



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DIPARTIMENTO DI INFORMATICA - SCIENZA E INGEGNERIA

Corso di laurea in Ingegneria Informatica

**MACHINE LEARNING SU DISPOSITIVI EDGE:
RICONOSCIMENTO FACCIALE TRAMITE
FRAMEWORK FLOWER**

RELATORE

Prof. Paolo Bellavista

CANDIDATO

Matteo Lombardi

Sessione Ottobre 2025

Anno Accademico 2024/2025

Indice

1	Machine learning su dispositivi a risorse limitate	3
1.1	Introduzione al machine learning	3
1.2	Machine learning applicato al riconoscimento facciale	8
1.3	Vantaggi e sfide del machine learning su edge computing	11
1.4	Tecniche di ottimizzazione di modelli di machine learning su dispositivi edge	14
2	Federated Learning: principi e implementazione con Flower	19
2.1	Introduzione al Federated Learning	19
2.2	Il framework Flower	25
2.3	Applicazioni e casi d'uso del federated learning	31
3	Implementazione di un sistema di riconoscimento facciale con Federated Learning	35
3.1	Obiettivi sperimentali	35
3.2	Architettura del sistema	36
3.3	Implementazione del sistema	37
3.4	Risultati sperimentali e sviluppi futuri	46

Introduzione

L'evoluzione tecnologica degli strumenti di intelligenza artificiale, con particolare riferimento al machine learning, ha favorito lo sviluppo di sistemi avanzati in grado di estrarre conoscenza utile e di supportare decisioni in molteplici contesti. Dalla medicina alla finanza, dalla sicurezza alla mobilità intelligente, tali soluzioni hanno dimostrato di poter incrementare l'efficacia delle attività umane e di affrontare problemi complessi.

Tradizionalmente, la realizzazione e l'utilizzo di sistemi di modelli di apprendimento automatico si sono basati sul paradigma del cloud computing, in cui tutti i dati e le elaborazioni vengono centralizzate per sfruttare infrastrutture ad alta potenza di calcolo. Questo approccio funziona perfettamente in numerosi contesti di applicazione, e ha permesso anche una rapida diffusione dell'utilizzo del ML anche ad utenti comuni. Negli ultimi anni però, sono emersi, in alcuni contesti applicativi, limitazioni significative delle architetture cloud, legate principalmente a sicurezza, consumo di banda e latenze troppo elevate. Diversi scenari come la guida autonoma e i servizi sanitari in tempo reale, nei quali certi requisiti sono fondamentali per un funzionamento accettabile, hanno spinto i ricercatori a trovare un'altra soluzione percorribile.

In questo contesto il paradigma dell'edge computing ha assunto sempre più rilevanza, spostando il carico computazionale dai data center centralizzati verso i dispositivi situati ai margini della rete. Smartphone, droni, videocamere e sensori IoT diventano i principali attori nei sistemi di apprendimento, non limitandosi a essere semplici generatori di dati, ma diventando piattaforme in grado di allenare ed eseguire modelli in maniera del tutto locale. All'interno di questo contesto si colloca il federated learning, un nuovo paradigma di apprendimento che ribalta lo schema tradizionale. Questo infatti sfrutta a pieno la natura distribuita di molti sistemi, spostando l'addestramento dei modelli dal server ai dispositivi terminali. Questo cambio di prospettiva porta con sé numerosi vantaggi in termini di privacy dei dati e scalabilità, diventando la soluzione preferita, o addirittura obbligata, in molti scenari. Dall'altro lato il federated learning, e più in generale l'edge computing, introduce anche nuove sfide non indifferenti, come quelle date da risorse computazionali limitate, autonomia energetica ridotta e addestramento in ambienti eterogenei.

La presente tesi si inserisce in questo scenario, con l'obiettivo di esplorare e implementare un sistema di riconoscimento facciale distribuito su dispositivi edge utilizzando il framework Flower, capace di simulare scenari complessi di questo tipo. Lo scopo finale del lavoro è quello di mostrare la fattibilità tecnica dell'approccio e ottenere un prototipo funzionante che possa essere utilizzato come base da ampliare per ricerca avanzata o contesti industriali.

Il testo è organizzato in cinque capitoli, ciascuno con un obiettivo ben delineato.

- Capitolo 1: Fornisce una panoramica della teoria alla base del machine learning

e lo applica a contesti edge, esplorando tecniche, vantaggi e sfide.

- Capitolo 2: Espone nel dettaglio il funzionamento del federated learning e il framework Flower, utilizzato a supporto del progetto.
- Capitolo 3: Tratta l'architettura e l'implementazione del sistema realizzato, analizzandone il funzionamento e le metriche prodotte.

Capitolo 1

Machine learning su dispositivi a risorse limitate

1.1 Introduzione al machine learning

Il mondo odierno, per via della digitalizzazione massiva e della diffusione di internet, è caratterizzato da un'enorme disponibilità di dati di ogni tipologia e struttura. Questo rappresenta un grande vantaggio per l'umanità, poiché tramite i dati è possibile comprendere il funzionamento di numerosi fenomeni e generare nuova conoscenza. Tuttavia, la capacità di apprendimento dell'essere umano è limitata, dato che nessun individuo può analizzare la mole di dati oggi disponibile nell'arco di una sola vita. Questo è il motivo per cui negli ultimi decenni è diventato sempre più importante sviluppare strumenti per la comprensione e l'estrazione di conoscenza dai dati. In questo contesto, l'intelligenza artificiale, e in particolare il machine learning, si è affermata come uno degli approcci più utili e interessanti, suscitando l'attenzione di numerosi ricercatori e aziende.

Il machine learning, o apprendimento automatico, si occupa della creazione di sistemi in grado di imparare autonomamente a svolgere un determinato compito e migliorarsi con l'esperienza, senza essere programmati specificatamente per farlo. Tali sistemi presentano alla base modelli matematici e statistici che, analizzando grandi quantità di dati, ricavano schemi generali utili per effettuare previsioni o prendere decisioni anche su casi mai osservati in precedenza. Questo approccio è utile nell'affrontare problemi complessi che risulterebbero difficili, se non impossibili, da risolvere con i metodi tradizionali di programmazione. Applicazioni di questo tipo si trovano in numerosi settori, dall'elaborazione del linguaggio naturale al riconoscimento di immagini, fino alla diagnosi in ambito medico. Per questo motivo, il progresso della ricerca nel machine learning è di fondamentale importanza per l'innovazione scientifica, economica e sociale.

L'efficienza di una soluzione di machine learning dipende principalmente da due fattori: i dati utilizzati per l'addestramento e l'algoritmo di apprendimento. Per quanto riguarda i dati, questi possono essere suddivisi in diverse categorie in base alla loro forma e al loro contenuto.

- **Dati strutturati:** Sono altamente organizzati e caratterizzati da una struttura ben definita che segue uno standard. Vengono solitamente organizzati in tabelle con righe e colonne, come accade per i database relazionali, e sono di facile accesso

e gestione. Rappresentano dati strutturati tabelle di database con informazioni su clienti (nome, età, indirizzo), registri di vendite o valori numerici raccolti da sensori.

- **Dati non strutturati:** Non presentano un formato e un'organizzazione pre-definita, rendendo così più complicata la loro interpretazione e gestione. Esempi di dati non strutturati sono immagini, video, file audio e testi liberi.
- **Dati semi-strutturati:** Non seguono uno schema rigido che può essere implementato in un database relazionale, ma presentano una certa organizzazione che li rende interpretabili. File HTML, JSON, XML e database NoSQL sono esempi di dati semi-strutturati.
- **Metadati:** Sono dati che descrivono altri dati, attribuendone significato aggiuntivo e fornendo informazioni utili per la loro interpretazione e utilizzo. Esempi di metadati di un file immagine sono la risoluzione, la dimensione e la data di creazione.

L'altro fattore che determina l'efficienza di una soluzione di apprendimento automatico è l'algoritmo utilizzato. Questo può appartenere a quattro categorie differenti: apprendimento supervisionato, non supervisionato, semi-supervisionato e per rinforzo. Ciascuna categoria si suddivide ulteriormente in specifiche famiglie di algoritmi, andando a formare un albero di classificazione utile per organizzare e comprendere la varietà di tecniche disponibili per il machine learning.

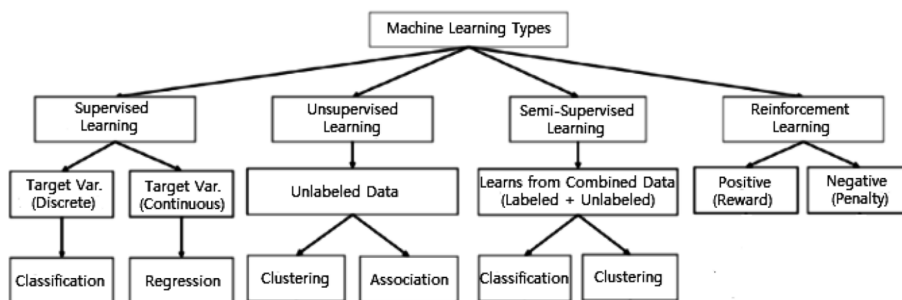


Figura 1.1: Albero di classificazione degli algoritmi di apprendimento automatico [1].

- **Apprendimento supervisionato:** Il modello viene addestrato tramite un dataset etichettato, cioè costituito da coppie input-output già note. L'obiettivo è quello di estrarre le regole generali che legano l'input all'output, così da poter prevedere correttamente il risultato per dati nuovi. Nella pratica la relazione input-output trovata viene incapsulata all'interno di una funzione $f(x)$ che produce un output y per ogni input x . I parametri del modello, ossia i coefficienti che compaiono nella funzione, vengono stimati sulla base dei dati presenti nel training set. Per fare ciò si definisce una misura di distanza tra le predizioni del modello e i dati forniti, chiamata funzione di perdita o funzione di costo, che viene minimizzata numericamente per determinare i valori ottimali dei parametri. Nell'addestramento di un modello funzionante occorre garantire due requisiti fondamentali:

1. Evitare l'underfitting, che si verifica quando si sceglie l'utilizzo di un modello troppo semplice per la complessità dei dati del problema. Questo non riesce quindi a catturare gran parte delle relazioni presenti tra input e output, non funzionando correttamente nè sul dataset di addestramento nè su nuovi dati.
2. Prevenire l'overfitting, ovvero la situazione in cui il modello ottenuto è eccessivamente complesso per il problema dato e finisce per adattarsi eccessivamente ai dati di addestramento. Un tale modello ottiene ottimi risultati sul training set ma non è in grado di generalizzare correttamente su dati nuovi.

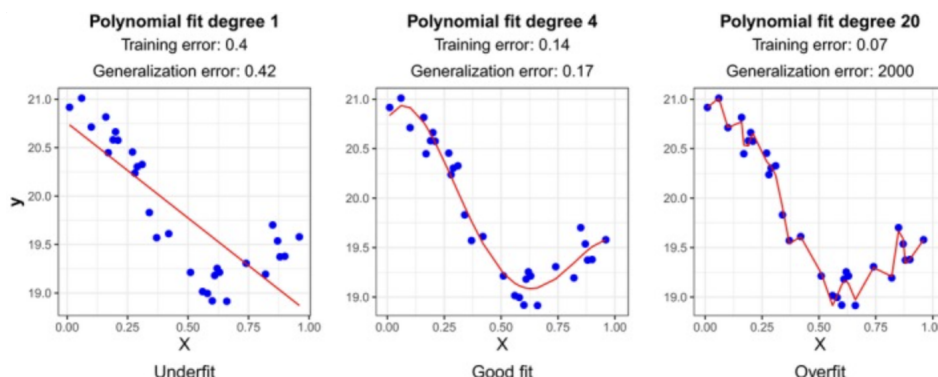


Figura 1.2: Underfitting e overfitting nell'apprendimento supervisionato [2].

- **Apprendimento non supervisionato:** Qui i dati non hanno etichette, ed è l'algoritmo di allenamento stesso che deve individuare in maniera autonoma relazioni, strutture e raggruppamenti. Questo tipo di apprendimento è ampiamente utilizzato per la creazione di modelli generativi, l'effettuazione di analisi esplorative su dati e la riduzione della dimensionalità.
- **Apprendimento semi-supervisionato:** Rappresenta una via di mezzo tra i due approcci precedenti, dato che vengono utilizzati sia dati etichettati che non. Nel mondo reale infatti dati con etichette sono rari e difficili da ottenere, mentre numerosi sono quelli senza di esse. Per questo motivo, soprattutto nei contesti in cui raccogliere etichette risulta oneroso, viene utilizzata questa tecnica di apprendimento, in cui si dispone di un piccolo insieme di dati etichettati e di una grande quantità di dati non etichettati. In questo modo si ottiene solitamente un modello migliore rispetto a quello ottenibile con l'uso dei soli dati etichettati.
- **Apprendimento per rinforzo:** Invece di utilizzare esempi di come il modello dovrebbe comportarsi, l'apprendimento si basa su un meccanismo di interazione con l'ambiente. Durante l'esecuzione delle azioni, l'agente riceve ricompense o penalità, con l'obiettivo finale di massimizzare il premio ottenuto. Questa tecnica è di grande importanza e utilità in contesti di automazione come la robotica o la guida autonoma. L'apprendimento per rinforzo può essere distinto in due grandi famiglie: model-based e model-free. Nel model-based viene creato un vero e proprio modello dell'ambiente in cui l'agente opera, che ne descrive il funzionamento e le ricompense associate alle azioni. In questo modo il sistema cerca di prevedere la dinamica del mondo esterno e pianifica la sequenza di azioni più adatta per massimizzare la propria ricompensa. Nell'approccio model-free invece l'agente non

conosce nè il funzionamento dell'ambiente su cui opera, nè come vengono distribuite le ricompense. L'apprendimento viene dunque svolto sfruttando solamente l'esperienza, ovvero modificando le proprie azioni in base al risultato dei tentativi precedenti per aumentare progressivamente il premio ottenuto.

Ciascuna delle categorie di apprendimento appena analizzate può essere implementata tramite diversi approcci proposti e studiati da ricercatori nel corso degli anni. La ricerca è tutt'ora molto attiva in questo ambito, e nuove tecniche continuano ad essere sviluppate e perfezionate con l'obiettivo di migliorare le prestazioni e l'applicabilità a scenari reali. Si presentano di seguito le tecniche più note e consolidate, al fine di fornire una panoramica dei principali paradigmi usati oggi nel machine learning.

- **Classificazione:** Metodo di apprendimento supervisionato in cui viene costruita una funzione f per assegnare a variabili di input x delle variabili di output y di tipo discreto. Viene utilizzata per distinguere dati in categorie predefinite, come nel riconoscimento facciale o nella diagnosi medica.
- **Regressione:** Simile alla classificazione, con la differenza che in output non si hanno categorie o etichette, ma una quantità continua. Modelli di questo tipo vengono impiegati in campi come la previsione finanziaria, la stima dei costi o la stima delle serie temporali.

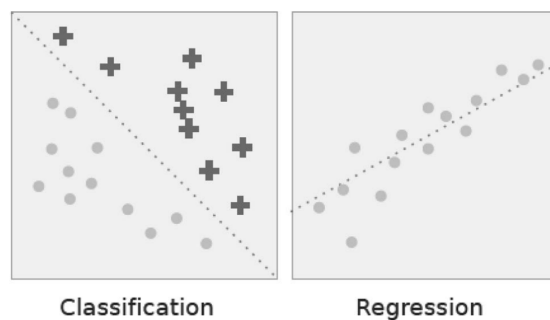


Figura 1.3: Classificazione vs. Regressione [1].

- **Clustering:** Tecnica di apprendimento non supervisionato che ha come obiettivo quello di identificare dei sottogruppi in un certo dataset senza conoscerli a priori. I dati vengono dunque raggruppati in categorie all'interno delle quali gli elementi risultano omogenei, massimizzando la somiglianza interna e la differenza tra gruppi distinti.
- **Riduzione della dimensionalità:** Nell'ambito del machine learning, uno spazio di dimensioni troppo elevate per i dati può rappresentare una sfida in termini di visualizzazione e analisi. Tale problema può essere mitigato tramite una tecnica di apprendimento non supervisionato, la quale trasforma ogni punto dati ad alta dimensionalità in due o più dimensioni, mantenendo la maggior parte delle informazioni relative. In questo modo si migliora l'interpretabilità umana dei dati, si riducono i costi computazionali e si evita l'overfitting dei modelli. Per ridurre la

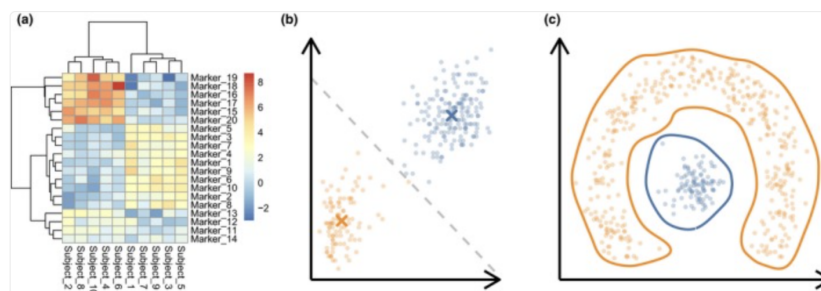
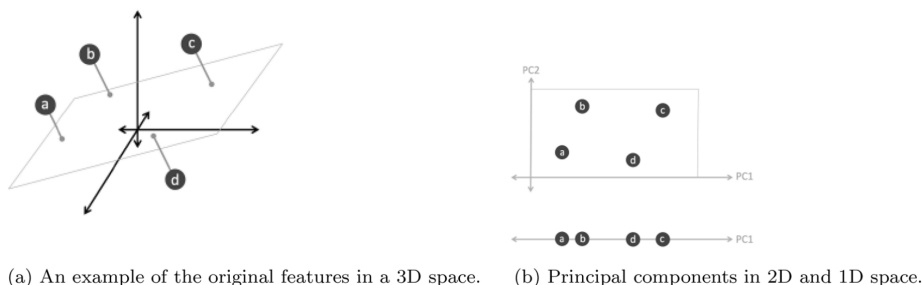


Figura 1.4: Alcuni approcci di clustering [2].

dimensionalità possono essere utilizzate due tecniche distinte: la selezione e l'estrazione delle caratteristiche. Nella selezione viene scelto un sottoinsieme delle variabili più informative, eliminando quelle ridondanti o poco rilevanti. L'estrazione invece genera nuove caratteristiche sintetiche che combinano o trasformano quelle esistenti, ottenendo così uno spazio di dimensioni inferiori.



(a) An example of the original features in a 3D space. (b) Principal components in 2D and 1D space.

Figura 1.5: Un esempio di riduzione della dimensionalità [1].

- Reti neurali e deep learning: Uno dei principali strumenti del machine learning, ispirate al funzionamento del cervello umano. Una rete neurale è composta da unità di elaborazione interconnesse, chiamate neuroni artificiali, organizzate su più livelli. Ciascun neurone riceve diversi input, li combina assegnando pesi che ne modulano l'importanza, per poi applicare una funzione di attivazione che produce l'output. I livelli su cui sono divisi i neuroni possono essere di input, output, o intermedi, anche detti nascosti. Sono questi ultimi che permettono alla rete di apprendere rappresentazioni sempre più complesse dei dati, aggiungendo livelli di astrazione. Il deep learning è un sottoinsieme delle reti neurali, caratterizzato dall'utilizzo di reti profonde, o DNN, ovvero con molti strati nascosti. È molto importante scegliere il numero appropriato di livelli intermedi, poiché all'aumentare della complessità cresce anche il tempo necessario per addestrare il modello.

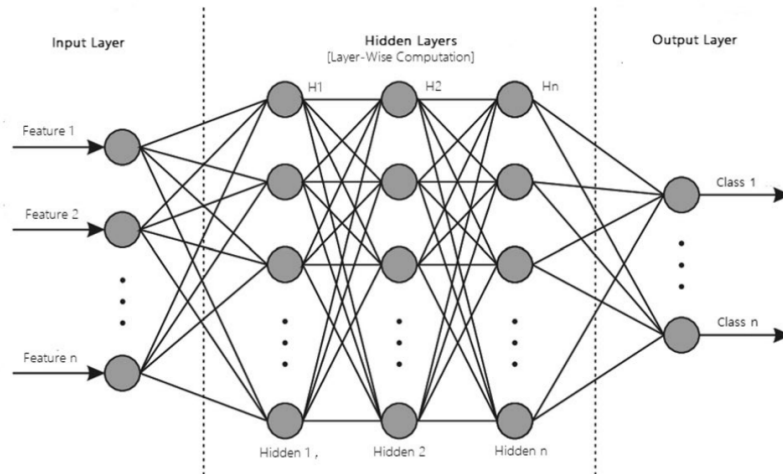


Figura 1.6: La struttura di una rete neurale profonda [1].

1.2 Machine learning applicato al riconoscimento facciale

Un ambito particolarmente interessante del machine learning è quello del riconoscimento facciale, che consiste nel riconoscere o verificare l'identità di una persona a partire dall'immagine o dal video del suo volto. Negli ultimi anni lo sviluppo dell'intelligenza artificiale ha consentito di migliorare l'efficacia di questa tecnologia e la conseguente applicazione in diversi settori. Il più rilevante è sicuramente quello della sicurezza, in cui l'utilizzo di fattori biometrici ha permesso di semplificare l'accesso a numerosi sistemi, richiedendo solamente la presenza dell'utente di fronte al sensore, senza la necessità di dover ricordare password o codici riservati.

Per un umano, identificare un volto è un compito piuttosto semplice e naturale, ma per una macchina risulta molto più complesso. Ciò è dato dalle numerose variabili insite ad un problema del genere, come le condizioni di illuminazione, variazioni nella posa, occlusione di parti del viso, espressioni facciali, bassa risoluzione dell'immagine, cambiamenti nei tratti del viso dovuti all'invecchiamento e così via. Per questo motivo, negli ultimi decenni sono stati proposti diversi approcci per risolvere il problema del riconoscimento facciale da prospettive differenti con l'obiettivo di aumentarne l'accuratezza.

Definiamo innanzitutto il problema e le caratteristiche in comune a tutte le soluzioni. Per essere utile, un sistema di face recognition deve soddisfare: capacità di lavorare sia con video che con immagini, di elaborare in tempo reale, di essere robusto in diverse condizioni di illuminazione, essere indipendente dalle caratteristiche delle persone (come età, etnia o sesso) e di poter lavorare con volti da diverse angolazioni. I sistemi di questo tipo sono solitamente costituiti dalla seguente pipeline di moduli:

1. **Rilevamento del volto**: Il processo inizia con la localizzazione di visi umani nell'immagine. Questo passaggio è necessario per determinare se l'immagine contiene volti e delimitare la regione da analizzare nelle fasi successive.

2. Allineamento del volto: L'immagine viene ridimensionata e ritagliata utilizzando una serie di punti di riferimento localizzati attraverso un rilevatore.

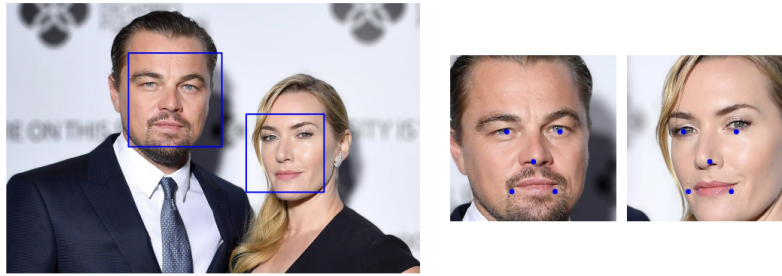


Figura 1.7: Esempio di allineamento del volto [3].

3. Estrazione delle caratteristiche: Si ricavano le informazioni distintive dei volti rilevati nei passaggi precedenti. Ciascun viso viene dunque rappresentato con un insieme di vettori di caratteristiche chiamati "firma", i quali descrivono i tratti principali, come bocca, naso e occhi.
4. Riconoscimento del volto: Qui, in base all'applicazione pratica che ha il sistema, viene svolta l'identificazione o la verifica del volto. Nel primo caso le caratteristiche estratte durante la fase precedente vengono confrontate con un database di persone con l'obiettivo di trovare la corrispondenza più probabile. La seconda opzione consiste invece nel confrontare i connotati del volto rilevato con quelli di uno specifico individuo al fine di accettare o respingere l'associazione.



Figura 1.8: Pipeline dei sistemi di riconoscimento facciale [3].

Il riconoscimento facciale, nel suo complesso, può essere dunque considerato un problema composito nell'ambito del machine learning, integrando tecniche di rilevamento, estrazione delle caratteristiche e classificazione.

Le prime ricerche sul riconoscimento facciale si sono focalizzate sull'utilizzo di metodi che cercavano la corrispondenza tra le caratteristiche principali della geometria di un volto, come la posizione e la grandezza degli occhi e della bocca. Questi metodi presentavano numerosi problemi e un'accuratezza molto bassa, funzionando correttamente solo in determinate condizioni, ma mostrarono per la prima volta che un computer fosse in grado di riconoscere dei visi. Successivamente vennero ideati metodi cosiddetti olistici, che utilizzavano come input l'intera superficie del volto, come la Principal Component Analysis [4] e la Linear Discriminant Analysis [5]. Allo stesso tempo un'altra tecnica che si stava sviluppando era quella degli approcci locali, i quali facevano uso di singole regioni del volto per trovare la corrispondenza. Questi due criteri vennero poi combinati insieme generando metodi ibridi, che costituiscono lo stato dell'arte fino

all'avvento delle tecniche basate sul deep learning, le quali hanno rivoluzionato il campo e raggiunto risultati superiori a quelli degli umani.

La tipologia più comune di metodi deep learning per il riconoscimento facciale sono le CNN [6], o reti neurali convoluzionali, un tipo specifico di rete neurale progettata per elaborare dati che hanno una struttura a griglia, come le immagini. Le CNN sono ampiamente utilizzate in questo ambito poiché, se addestrate su dataset molto ampi, riescono ad estrarre caratteristiche precise dei volti analizzati, gestendo anche variazioni di espressione, posa, illuminazione e occlusioni senza dover essere progettate specificamente per farlo. Sono tre i fattori principali che influenzano l'accuratezza dei metodi basati su CNN per il riconoscimento facciale: dati di addestramento, architettura della CNN e funzione di perdita. Per quanto riguarda il dataset usato per l'addestramento, è importante che questo sia ampio e profondo. L'ampiezza consiste nel contenere un numero elevato di identità diverse, chiamate tecnicamente classi. In questo modo si ottiene un'alta variabilità inter-classe che consente di creare un sistema in grado di riconoscere anche persone non presenti nel training. La profondità del dataset, ossia la presenza di molte immagini per singola classe, permette invece al modello di identificare la stessa persona in condizioni diverse. L'architettura e la funzione di perdita utilizzata per l'addestramento possono variare da modello a modello e la loro scelta dipende solitamente dai vincoli applicativi, in quanto vi è un trade-off tra precisione, risorse computazionali e tempi di inferenza. Per il face recognition moderno le architetture basate su varianti di ResNet [7] e i loro miglioramenti sono spesso la soluzione di riferimento. Queste consentono infatti di evitare un importante problema delle reti neurali, le quali aumentando di molto in profondità presentano un peggioramento nelle prestazioni, causato principalmente dalla difficoltà del gradiente di propagarsi correttamente attraverso molti livelli. Le ResNet affrontano questa limitazione introducendo il concetto di residual block, in cui l'input viene collegato direttamente all'uscita tramite una scorciatoia, consentendo così di poter addestrare reti molto più profonde e quindi complesse.

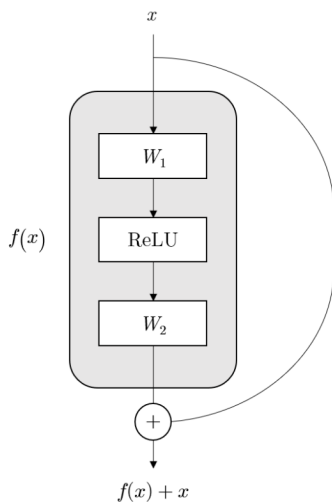


Figura 1.9: Concetto di residual block nelle architetture ResNet [3].

L'ultimo fattore di grande importanza nella riuscita di un buon modello di riconoscimento facciale è quello della scelta della funzione di perdita, o loss function. Quest'ul-

tima discrimina la strategia di addestramento che si decide di adottare, che in questo caso viene solitamente scelta tra due tipologie differenti. Un primo approccio consiste nel trattare il problema come una classificazione, considerando ciascun individuo come una classe distinta. In questo modo la rete viene inizialmente addestrata per distinguere tra i soggetti presenti nel dataset di training. Dopo l'addestramento, però, quello che interessa non è la classificazione svolta dal modello, ma le rappresentazioni digitali dei volti, chiamate bottleneck features, le quali vengono estratte dal penultimo livello della rete, quello precedente al livello di classificazione. In questo modo, eliminando l'ultimo strato dell'architettura neurale, è possibile utilizzare il sistema appena allenato anche su volti nuovi. Una seconda strategia di apprendimento è quella del metric learning, la quale mira ad imparare uno spazio di rappresentazione in cui i volti della stessa persona si trovano vicini e volti di persone distinte sono lontani. Quest'ultimo approccio è più efficace nel generalizzare a individui non presenti nel dataset di addestramento.

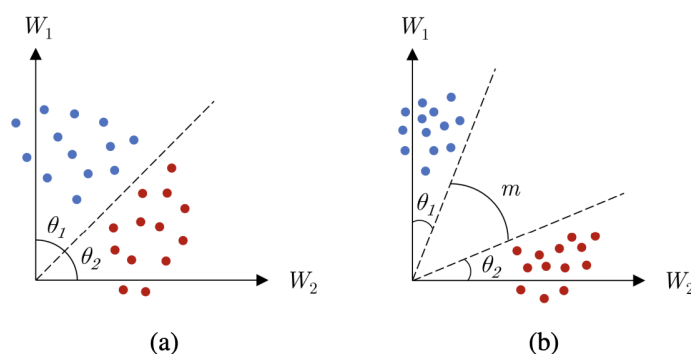


Figura 1.10: Esempio di introduzione di margine tra due classi nel metric learning [3].

1.3 Vantaggi e sfide del machine learning su edge computing

Gran parte dei servizi di machine learning odierni funzionano utilizzando architetture di tipo cloud. In questo modo gli utenti, spesso anche piccole e medie imprese, non hanno bisogno di affrontare i costi per l'acquisto e il mantenimento di hardware adatto all'allenamento ed esecuzione dei modelli. Questi ultimi necessitano infatti di un'elevata potenza di calcolo, e la loro gestione centralizzata permette il noleggio di risorse computazionali in maniera economica e scalabile. Inoltre proprio la natura dell'addestramento di alcuni sistemi di machine learning necessitano l'accesso a grandi dataset, il che è facilitato utilizzando architetture cloud che consentono di aggregare e archiviare dati provenienti da fonti diverse. Tuttavia, questo approccio non è adatto a soddisfare alcuni requisiti fondamentali richiesti in specifici ambiti di applicazione. Per esempio, alcuni contesti richiedono velocità di risposta estremamente elevate per funzionare correttamente, come ad esempio quello della guida autonoma. Qui, considerando anche l'ambiente complicato in cui guida un veicolo, ritardi nelle risposte possono portare a decisioni sbagliate con conseguenze gravi. Un altro punto di debolezza del cloud computing riguarda la privacy e la sicurezza. Architetture di questo tipo obbligano infatti l'utente a trasferire i propri dati su server spesso lontani, aumentando la possibilità

di furti o accessi non autorizzati. Nel caso dell'utilizzo di dati riservati o connessi a grandi organizzazioni questo rischio non può essere tollerato, e risulta quindi necessario adottare soluzioni alternative. Negli ultimi anni sta diventando sempre più popolare un altro paradigma informatico che non sostituisce le architetture di tipo cloud ma le complementa, andando a colmare alcune delle sue limitazioni. Si tratta dell'edge computing, che consiste in un modello architetturale in cui il calcolo e l'analisi dei dati vengono spostati vicino alla loro fonte, ossia ai margini della rete. Per margini si intende qualsiasi risorsa di elaborazione e di rete tra la fonte dei dati e il cloud, come smartphone, router, gateway, e micro-data center. L'architettura dell'edge computing si sviluppa solitamente tramite una struttura a tre livelli, come mostrato in figura 1.11.

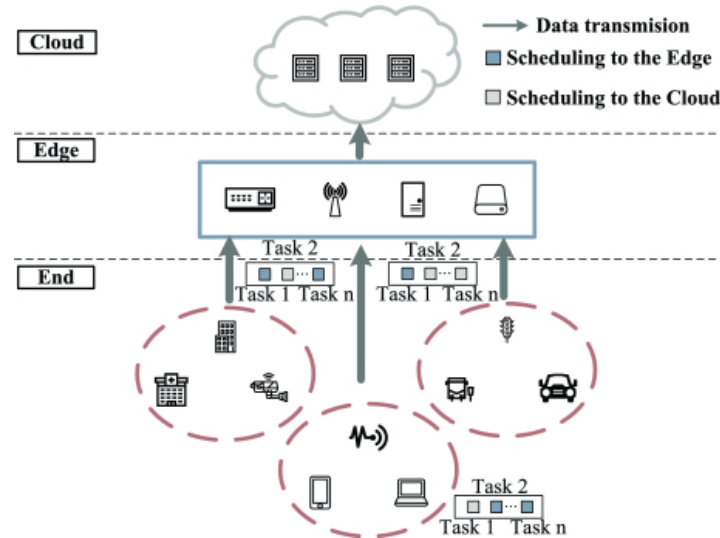


Figura 1.11: Architettura dell'edge computing [8].

Il primo livello è quello dei dispositivi endpoint, ovvero l'insieme dei sensori e apparati intelligenti che si trovano a diretto contatto con l'ambiente fisico. Questi strumenti hanno il compito di raccogliere i dati dal mondo esterno e, in alcuni casi, sono anche in grado di svolgere una prima fase di elaborazione locale utilizzando algoritmi o modelli di machine learning leggeri. A questo livello troviamo dispositivi come smartphone, videocamere di sorveglianza, droni, dispositivi IoT e wearable devices. Il secondo livello è quello dei nodi edge, ovvero elementi intermedi come gateway, router o micro-data center situati in prossimità geografica degli endpoint. Qui avviene la gestione e l'analisi dei dati provenienti dai dispositivi vicini, rendendo possibile l'esecuzione di modelli di intelligenza artificiale pre-addestrati senza il trasferimento continuo di grandi quantità di dati verso il cloud. Il terzo e ultimo livello rimane quello del cloud computing, in cui i data center centralizzati mantengono un ruolo fondamentale svolgendo compiti che necessitano una grande potenza di calcolo, come l'addestramento di modelli complessi di machine learning e l'analisi dei dati raccolti in maniera distribuita. Il cloud può fungere anche da punto di macro-controllo dell'intera architettura, coordinando il corretto funzionamento e l'aggiornamento dei modelli utilizzati nei nodi edge.

L'adozione di questo nuovo paradigma informatico sposta gran parte del carico computazionale e decisionale vicino alla fonte dei dati, offrendo una serie di benefici.

- **Riduzione della latenza:** Essendo il livello edge molto più vicino alla sorgente

dei dati, la distanza di trasmissione viene notevolmente accorciata, ottenendo una riduzione del tempo di trasmissione. In questo modo la risposta arriva più rapidamente e, oltre a un miglioramento delle prestazioni si ha anche una maggiore sicurezza operativa.

- Ottimizzazione della banda di rete: In alcuni scenari caratterizzati da una generazione continua di dati provenienti da svariati dispositivi, il trasferimento di questi al cloud può generare congestioni di reti e problemi di disponibilità del servizio. L'edge computing riduce i costi in termini di comunicazione filtrando e pre-elaborando a livello edge i dati localmente, inviando al cloud i risultati finali dell'operazione e riducendo dunque il traffico di rete.
- Sicurezza e privacy: L'elaborazione dei dati direttamente sul dispositivo di raccolta o sui nodi edge implica una minore, in alcuni casi nulla, quantità di dati sensibili trasferiti al cloud. In questo modo si preserva la privacy e viene ridotto il rischio di intercettazioni non autorizzate, due requisiti fondamentali in ambiti come la sanità e la finanza, dove spesso è obbligatorio operare in questo modo per essere conformi a normative stringenti come il GDPR.

Nonostante i numerosi vantaggi appena elencati, l'utilizzo su larga scala dell'edge computing presenta alcune limitazioni e sfide che sono tutt'ora oggetto di ricerca per aumentarne la diffusione e l'applicabilità. Una questione molto importante riguarda l'offloading, ovvero il trasferimento parziale o totale delle operazioni di elaborazione dai dispositivi periferici ai nodi edge o al cloud. Questo viene svolto per superare il problema delle limitazioni di calcolo e batteria dei dispositivi terminali, ma porta con sé una serie di complicazioni che rendono di fondamentale importanza la domanda su quanti e quali compiti delegare. Tale decisione dipende da molti fattori che variano in base alla tipologia di problema e dei dispositivi adottati, come la potenza di calcolo disponibile su questi, lo stato della rete, la dimensione dei dati e i requisiti temporali dell'applicazione. In letteratura l'offloading è spesso considerato come un problema di ottimizzazione, il quale cerca di bilanciare qualità del servizio, consumo energetico e latenza. Per comprendere meglio il concetto prendiamo in esempio l'inferenza su DNN, la quale può essere eseguita esclusivamente su dispositivi mobili oppure interamente delegata al server cloud/edge. Nel primo caso, se viene utilizzata una rete neurale con molti livelli, il tempo di inferenza sarà decisamente maggiore per via della ridotta capacità computazionale. D'altra parte, un server impiegherà molto meno per svolgere lo stesso compito, ma il tempo di risposta dipenderà principalmente dalla latenza di trasmissione dei dati, che può variare ampiamente in base al contesto di applicazione. Un'altra questione importante riguarda la gestione delle risorse limitate sui dispositivi periferici e la sicurezza di questi. Se da una parte con il mantenimento dei dati sugli apparati terminali vengono evitati i problemi di banda e privacy discussi in precedenza, dall'altra la natura eterogenea e distribuita di questi rappresenta un problema per la gestione delle risorse e degli attacchi informatici. È necessario infatti adottare una soluzione intelligente ed efficiente per la gestione della memoria e capacità computazionale dei nodi terminali. Inoltre l'utilizzo di questo paradigma introduce una superficie di attacco molto più ampia, rendendo difficile garantire un livello di protezione adeguato in una rete così decentralizzata ed eterogenea, in cui le soluzioni tradizionali per la sicurezza dei dati e i problemi di privacy del cloud computing non sono applicabi-

li. Ulteriore sfida del machine learning su edge computing riguarda l'ambito dell'edge learning. Quest'ultimo consiste nell'addestrare modelli ML direttamente sui dispositivi edge, riducendo o eliminando la dipendenza dall'infrastruttura cloud. Alcuni sistemi richiedono infatti un apprendimento locale di questo tipo al fine di tutelare la privacy dei dati utilizzati e ottenere un modello personalizzato per casi d'uso specifici. L'edge learning rappresenta tuttavia una sfida non trascurabile, vista soprattutto la natura dei dispositivi utilizzati. L'addestramento di un modello di machine learning è infatti un processo solitamente oneroso sotto il profilo computazionale ed energetico, richiedendo una grande quantità di operazioni matematiche complesse. I dispositivi periferici utilizzati per l'edge learning presentano però molti limiti per quanto riguarda efficienza computazionale e batteria, causando potenzialmente un tempo di addestramento troppo elevato. Questi strumenti sono inoltre caratterizzati da una memoria limitata e generano dati spesso non etichettati, il che rende difficile soddisfare il grande fabbisogno di dati strutturati richiesto dai modelli ML.

I problemi e le sfide per l'utilizzo del machine learning in architetture di tipo edge sono numerose e complicate. Tuttavia è molto importante che i ricercatori continuino a studiare e sviluppare nuove soluzioni per progettare e adottare metodi efficienti e scalabili, in quanto significativi e numerosi sono anche gli ambiti di applicazione. Un esempio è quello delle smart city, ovvero centri abitati che utilizzano mezzi informatici per migliorare l'efficienza energetica, alleggerire il traffico, garantire la sicurezza e aumentare la qualità della vita. Qui la grande quantità di dati da analizzare rende necessario l'utilizzo di AI. Inoltre, soprattutto per quanto riguarda sicurezza e sanità, l'edge computing diventa il paradigma da utilizzare per via dei requisiti di bassa latenza e sicurezza dei dati. Altro dominio di applicazione sempre più rilevante è quello dell'Internet of Vehicles. Il settore automobilistico intelligente richiede infatti tempi di risposta minimi per applicazioni come guida autonoma, infotainment e sicurezza stradale. L'EC fornisce capacità di calcolo distribuite vicino ai veicoli, necessarie in ambienti che si muovono ad alta velocità.

1.4 Tecniche di ottimizzazione di modelli di machine learning su dispositivi edge

Nella sezione precedente sono state analizzate le numerose sfide e difficoltà nell'utilizzo del machine learning su edge computing. La potenziale utilità di questo approccio è infatti bilanciata da una forte complessità dovuta ai vincoli computazionali, di memoria ed energetici che caratterizzano i dispositivi usati in questo campo. Tuttavia la ricerca è molto attiva, soprattutto negli ultimi anni, nel cercare soluzioni che rendano sempre più adottabili sistemi di questo tipo.

Il primo aspetto che vediamo nel dettaglio è quello dell'offloading, che come descritto in precedenza riguarda la decisione sulla delega parziale o totale delle operazioni di elaborazione dai dispositivi periferici ai nodi edge o al cloud. Nonostante tutti i vantaggi già discussi che si ottengono dall'esecuzione di modelli ML direttamente su dispositivi periferici, non è sempre necessario farlo. Per questo è importante capire quando e cosa delegare, per ottenere il giusto compromesso tra capacità di calcolo, precisione, energia e latenza. Possiamo distinguere quattro scenari principali di offloading.

1. Offloading classico: Si basa sulla decisione binaria di eseguire il modello DNN direttamente sul dispositivo mobile oppure delegare completamente il compito all'edge server o al cloud. Molte tecniche che consentono di prendere questa scelta in maniera intelligente si basano sull'utilizzo di algoritmi di intelligenza artificiale. Approcci recenti, come DeepDecision [9], mirano a ridurre uno specifico fattore che influenza la scelta oppure più elementi contemporaneamente. Il bilanciamento simultaneo di due o più obiettivi spesso contrastanti, come il consumo di energia e la latenza, viene affrontato solitamente con l'utilizzo di metodi di reinforcement learning. Tecniche recenti usano nello specifico algoritmi di deep reinforcement learning (DRL), che possiede una maggiore capacità di gestire spazi di stato ad alta dimensionalità per dinamiche di rete complesse. Va tuttavia osservato come l'utilizzo del DRL necessiti la presenza di capacità computazionali significative da parte dei dispositivi edge, ipotesi poco realistica nella pratica. Per questo approcci avanzati si basano su strategie di offloading distribuito con modelli DRL multi-agente.
2. DNN Model Partitioning: Questo metodo esegue parte del lavoro sui nodi terminali e la restante sui server edge o sul cloud. Per fare ciò sfrutta la struttura a livelli delle DNN, effettuando una divisione in due parti che vengono eseguite su componenti diversi dell'architettura. L'intuizione dietro questo criterio si basa sul fatto che dopo il calcolo dei primi strati del modello DNN, la dimensione dei risultati intermedi è relativamente piccola, rendendo la trasmissione in rete a un server edge più leggera e quindi veloce rispetto ai dati grezzi originali. L'approccio Neurosurgeon [10] ad esempio decide in maniera intelligente a che livello partizionare la DNN, tenendo conto anche delle condizioni della rete. Oltre a una partizione di questo tipo è possibile suddividere i dati di input, permettendo di adattarsi alla capacità di memoria dei dispositivi edge.
3. Dispositivi edge con cloud: Viene svolto un uso ponderato delle potenti risorse di elaborazione nel cloud delegando a questo parte dell'elaborazione. Un approccio di questo tipo molto utilizzato consiste nell'utilizzo del DNN Model Partitioning tra livello edge e cloud, dividendo il modello in due parti. Il server edge riceve i dati in input, esegue l'elaborazione dei livelli inferiori e invia i risultati intermedi al cloud. Questo elabora i livelli superiori e invia i risultati ai dispositivi terminali. In aggiunta a ciò viene utilizzata in alcuni casi la tecnica del fast exiting [11], che consiste nell'introdurre dei punti di uscita anticipata in corrispondenza di alcuni layer intermedi della rete. Se questi rispettano dei criteri di confidenza si può interrompere l'inferenza ed emettere direttamente l'output, diminuendo il tempo di risposta. Un altro metodo utilizzato per velocizzare il processo di elaborazione è quello del Precog [12], che si basa sull'intuizione che nell'edge computing, soprattutto nell'ambito della classificazione di immagini, i dispositivi serviti da uno stesso edge server sono geograficamente vicini tra loro e tendono a generare input simili. Viene dunque svolta una specializzazione locale dei modelli, fornendo a ogni gruppo di dispositivi un modello leggero specializzato sugli input più probabili in quella zona. Se la classificazione sul dispositivo fallisce, la richiesta viene inviata al server che mantiene al suo interno un modello più generale.

4. Calcolo distribuito: Invece di spostare il carico computazionale dai dispositivi terminali ad altri dispositivi più potenti, si utilizza una tecnica di lavoro distribuita, dove i calcoli DNN vengono spartiti tra più dispositivi edge, anche detti helper. La decisione sulla partizione della rete neurale viene svolta prendendo in considerazione la capacità di calcolo e/o la memoria dei dispositivi finali. I dati di input vengono invece distribuiti agli helper cercando di bilanciare il carico, e tale scelta può essere adattata in fase di esecuzione per tener conto di variazioni delle risorse o delle condizioni della rete.

Il processo di offloading richiede del tempo nell'invio dei dati attraverso la rete dai dispositivi finali ai server edge o cloud. È possibile comunque adottare tecniche atte a migliorare l'efficienza di questa fase e ridurre i tempi di comunicazione. Il preprocessing locale dei dati può risultare utile in molti casi, e consiste nello svolgere operazioni sull'apparato periferico prima dell'invio dei dati al server. Tra le tecniche più utilizzate rientrano ad esempio la rimozione delle ridondanze e dei dati poco significativi, come per frame identici all'interno di un video o immagini sfocate. Inoltre, dato che in genere uno stesso server edge deve poter servire più richieste provenienti da diversi dispositivi, è importante adottare una buona gestione delle risorse anche a questo livello dell'architettura. Qui infatti troviamo dispositivi con una maggiore potenza rispetto a quella degli strumenti terminali, ma comunque non abbastanza da essere equiparati ai server di tipo cloud. Ottimizzazioni dell'utilizzo delle risorse includono l'adattamento in tempo reale alle condizioni variabili della rete e del carico computazionale, riservando a tempo di esecuzione la scelta sull'utilizzo di una versione più veloce o più accurata in funzione delle risorse disponibili. Un altro approccio è quello del transfer learning, che consiste nell'impiego, all'interno della stessa architettura edge, di applicazioni che svolgono funzionalità diverse ma che condividono i livelli inferiori della rete neurale, catturando così pattern di base comuni e sfruttando la ridondanza tra compiti diversi.

Nonostante l'offloading sia molto popolare nell'edge computing, in alcuni casi è necessario o conveniente effettuare l'intera esecuzione direttamente nei dispositivi terminali, caratterizzati da forti limiti nelle risorse. In questo caso è importante adottare accorgimenti per ridurre il tempo di esecuzione e ottenere un sistema che funzioni correttamente e rispetti le specifiche. Un modo per fare ciò consiste nel progettare modelli leggeri, ovvero caratterizzati da un numero limitato di parametri. Così facendo viene ridotta la memoria necessaria e la latenza, mantenendo comunque un'elevata accuratezza. Modelli di questo tipo sono caratterizzati da convoluzioni più semplici, che riducono di conseguenza il numero di moltiplicazioni, o dall'unione di più fasi di lavoro insieme. Un altro approccio è quello di comprimere modelli già esistenti per consentire la loro esecuzione sui dispositivi edge con una perdita di precisione minima. Le principali tecniche includono:

- Quantizzazione: I parametri vengono compressi passando da numeri in virgola mobile a numeri con bassa larghezza di bit, evitando così costose moltiplicazioni in virgola mobile.
- Pruning: Vengono rimossi i parametri meno importanti, come quelli con un valore vicino a 0.
- Knowledge distillation: Viene creata una DNN più leggera che imita il comportamento di una DNN più grande e potente. Questo viene fatto addestrando la

DNN più piccola sugli output generati dalla DNN più grande, approssimando così la sua funzione.

Oltre ad approcci di tipo software per migliorare l'inferenza di modelli di deep learning su dispositivi edge, stanno diventando sempre più popolari soluzioni che fanno riferimento ad hardware specializzato. Un esempio è quello delle Tensor Processing Unit (TPU) [13] di Google, ovvero circuiti integrati sviluppati appositamente per eseguire le operazioni matematiche tipiche delle reti neurali in maniera estremamente efficiente. Altri sistemi si basano su un accesso ottimizzato alla memoria, come il progetto ShinDianNao [14], che riduce i movimenti di dati e migliora l'efficienza energetica. Infine, molte delle aziende più celebri per lo sviluppo di microprocessori e schede grafiche, come Intel, Nvidia e Qualcomm, stanno sviluppando interi ecosistemi hardware-software adatti all'esecuzione accelerata di sistemi di machine learning su dispositivi mobili e embedded.

Infine, un ulteriore ambito dell'edge computing che necessita di ottimizzazioni e strategie dedicate per funzionare correttamente è quello dell'edge learning. Come discusso in precedenza infatti, l'addestramento locale di modelli, nonostante sia di forte utilità in diversi scenari, rappresenta una sfida non trascurabile per via delle risorse limitate. Un primo tipo di metodi di addestramento edge è rappresentato dalle tecniche distribuite e collaborative. Queste funzionano sfruttando le capacità di calcolo di più dispositivi edge e aggregando i loro risultati. Un esempio è quello del federated learning [15], che prevede l'addestramento di un modello condiviso tra diversi dispositivi edge i quali provvedono al suo aggiornamento in maniera iterativa sulla base dei dati locali, con l'obiettivo di preservare la privacy di questi. Diversi modelli vengono dunque allenati su svariati dispositivi e solamente i risultati finali vengono inviati a un server centrale, che svolge un lavoro di aggregazione formando il modello finale senza avere accesso ai singoli dati utilizzati. Un altro esempio è quello dello split learning [16], che agisce dividendo il modello in più sezioni, ciascuna delle quali viene addestrata su un client diverso e, invece di trasferire dati grezzi, invia solo i pesi dell'ultimo livello di ciascuna sezione al client successivo. Un secondo gruppo di metodi utilizza modelli pre-addestrati adattandoli al caso specifico, senza doverli creare da zero. Il transfer learning [17] ad esempio riutilizza parti di modelli già addestrati per risolvere un determinato problema, adattandole a un nuovo compito differente ma strettamente correlato. L'incremental learning [18] utilizza invece un modello che viene aggiornato e ampliato man mano che nuovi dati diventano disponibili. Il meta-learning [19] è un ulteriore metodo adattivo che si concentra sull'utilizzare reti addestrate ad apprendere nuovi compiti in maniera rapida ed efficace. In questo modo è possibile adattare rapidamente il sistema a nuovi contesti esponendolo a una quantità di dati minima.

Capitolo 2

Federated Learning: principi e implementazione con Flower

2.1 Introduzione al Federated Learning

Il federated learning è un approccio di edge learning distribuito che consente di addestrare un modello sui dati di più dispositivi locali, anche detti client, senza dover inviare questi ultimi a un server centrale. Questo tipo di apprendimento inverte la metodologia di addestramento tradizionale, la quale richiede la raccolta centralizzata dei dati in un server, evitando così problemi di privacy, banda e regolamentazioni. Nella pratica il funzionamento si compone di diversi step:

1. Inizializzazione del modello globale: Vengono inizializzati i parametri del modello in modo causale o da un checkpoint salvato in precedenza, come accade per l'apprendimento centralizzato classico.
2. Invio del modello ai client: I parametri del modello globale vengono inviati a tutti i nodi client garantendo una base di partenza comune.
3. Addestramento locale dei modelli: Ciascun client svolge un piccolo training locale del modello su un proprio set di dati.
4. Invio degli aggiornamenti al server: I client inviano i modelli aggiornati al server trasmettendo i parametri finali o i gradienti accumulati durante l'addestramento.
5. Aggiornamento del modello globale: Il server riceve gli aggiornamenti del modello dai nodi client, ottenendo versioni diverse per via dei distinti dataset utilizzati. Per ottenere un modello globale aggiornato occorre quindi combinare tutti i risultati tramite un processo chiamato aggregazione.

Questi passaggi costituiscono un singolo ciclo di federated learning, e si ripetono in maniera iterativa per arrivare a ottenere un modello completamente addestrato.

Nonostante la struttura di base sia sempre costituita dai passaggi appena descritti, esistono diverse metodologie per implementare il federated learning, che variano principalmente per l'algoritmo di aggregazione adottato. Quest'ultimo infatti influenza le performance del sistema di addestramento, poiché determina in che modo ciascun

contributo dei client viene aggiunto al modello globale. In generale l'obiettivo di un algoritmo di federated learning è quello di ottimizzare la seguente funzione

$$\min_w F(w) = \sum_{k=1}^K p_k F_k(w) \quad (2.1)$$

dove w sono i pesi del modello globale, $F(w)$ è la funzione di perdita globale, $F_k(w)$ la funzione di perdita locale del client k e p_k il peso del client k nel calcolo della media, tipicamente proporzionale alla dimensione del dataset locale. In altre parole, le diverse tecniche di federated learning mirano a trovare un modello w che minimizzi la loss globale, cioè una combinazione pesata delle loss locali di tutti i client.

Oltre a questo obiettivo primario, diverse altre sono le sfide da affrontare nella creazione di un algoritmo di federated learning. Una prima problematica è quella dell'eterogeneità. I dati utilizzati per svolgere l'allenamento possono variare significativamente tra i vari client, dato che ciascuno raccoglie dati dal proprio ambiente locale. La distribuzione complessiva risulta dunque non indipendente e identicamente distribuita (non-IID). Anche i modelli sono eterogenei, dato che in contesti reali alcuni client possono preferire modelli personalizzati per migliorare il funzionamento nella loro area di competenza. Pertanto risulta difficile trovare un algoritmo di aggregazione adatto a modelli differenti tra loro. L'eterogeneità dei sistemi è un ulteriore aspetto da considerare, dato che i client possono disporre di risorse hardware molto diverse. Si individuano solitamente dispositivi più lenti che rallentano l'intero processo di addestramento, i cosiddetti stragglers. Occorre dunque decidere se attendere tutti i partecipanti per migliorare l'accuratezza oppure scartare i più lenti attribuendo più importanza alla velocità di convergenza. Un'ulteriore sfida riguarda la protezione della privacy. Uno dei principali motivi che portano all'utilizzo del federated learning è proprio la protezione della riservatezza dei dati, e questa è garantita dal principio alla base di questo approccio, che mantiene i dataset sui dispositivi locali e condivide solo i parametri del modello. Tuttavia, è stato dimostrato che anche questi parametri possono rivelare informazioni sensibili sugli utenti tramite attacchi di inferenza o ricostruzione dei dati. Per questo motivo è importante non trascurare l'aspetto e adottare tecniche di sicurezza avanzate durante la trasmissione.

Uno degli algoritmi più famosi e utilizzati nell'ambito del federated learning è il Federated Averaging (FedAvg) [15]. È stato introdotto nel 2017 ed è il primo metodo pratico e scalabile per addestramenti di tipo distribuito, rappresentando una base per gli sviluppi e le ricerche future. Il FedAvg si basa sullo svolgimento di round, ossia cicli iterativi che vengono ripetuti per ottenere la convergenza del modello globale. Ad ogni round il server centrale sceglie, casualmente o secondo dei criteri, un sottoinsieme dei client disponibili, i quali ricevono una copia del modello globale corrente w_t . Ogni client esegue dunque un addestramento locale del modello sui propri dati, utilizzando lo Stochastic Gradient Descent (SGD) [20], ossia un algoritmo di ottimizzazione che ha come obiettivo quello di trovare i valori dei pesi del modello che minimizzano la funzione di perdita. Per fare ciò viene calcolato il gradiente, ossia la direzione in cui modificare tali pesi per ridurre l'errore. Nel FedAvg, dato che i tempi di comunicazione al server sono solitamente i più onerosi, ciascun client può svolgere più passate di addestramento locale sui propri dati prima di inviare i nuovi parametri. In maniera formale, se un client k esegue E passate locali ad un learning rate η , si ottiene il seguente modello

aggiornato w_k^{t+1}

$$w_k^{t+1} = w_k^t - \eta \nabla F_k(w_k^t) \quad (2.2)$$

Una volta terminati tutti gli aggiornamenti locali, i client inviano i parametri ottenuti al server, il quale aggrega i modelli tramite una media pesata. Tali pesi dipendono esclusivamente dal numero di esempi presenti nei dataset di allenamento, dunque l'utilizzo di più dati corrisponde a una maggiore influenza nell'aggiornamento del modello globale. La funzione di aggregazione di un singolo round risulta dunque essere la seguente

$$w^{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_k^{t+1} \quad (2.3)$$

dove K è il numero di client utilizzati in quel round, n_k è il numero di esempi nel dataset del client e $n = \sum_{k=1}^K n_k$ è il totale degli esempi nei client selezionati. È possibile distinguere tre parametri che controllano l'algoritmo FedAvg:

- C : frazione di client selezionati per round. Un numero elevato comporta una maggiore rappresentatività della distribuzione dati, aumentando l'accuratezza del modello globale. D'altra parte aumenta il carico di calcolo e di comunicazione, causando problemi in ambienti a banda limitata.
- E : numero di passate locali effettuate da ciascun client in un round. Un maggior numero di epoche permette meno comunicazioni, tuttavia se i dati sono molto eterogenei tra client diversi si rischia di allontanare troppo i modelli locali da quello globale, peggiorando le prestazioni.
- B : dimensione dei minibatch, ovvero sottoinsiemi del dataset, utilizzati per l'addestramento. Valori piccoli causano rumore nel gradiente che aiuta il modello a non bloccarsi in minimi locali e a generalizzare meglio, a discapito però del maggior numero di aggiornamenti interni necessari per la convergenza. Batch grandi riducono la capacità di generalizzare ma risultano in addestramenti più veloci.

Esperimenti condotti con FedAvg hanno evidenziato che all'aumentare del numero di aggiornamenti locali (riducendo B o aumentando E) vi è una forte riduzione del numero di round necessari per raggiungere un certo livello di accuratezza. In questo modo diminuiscono il numero di comunicazioni al server, rendendo questa tecnica una delle più efficaci.

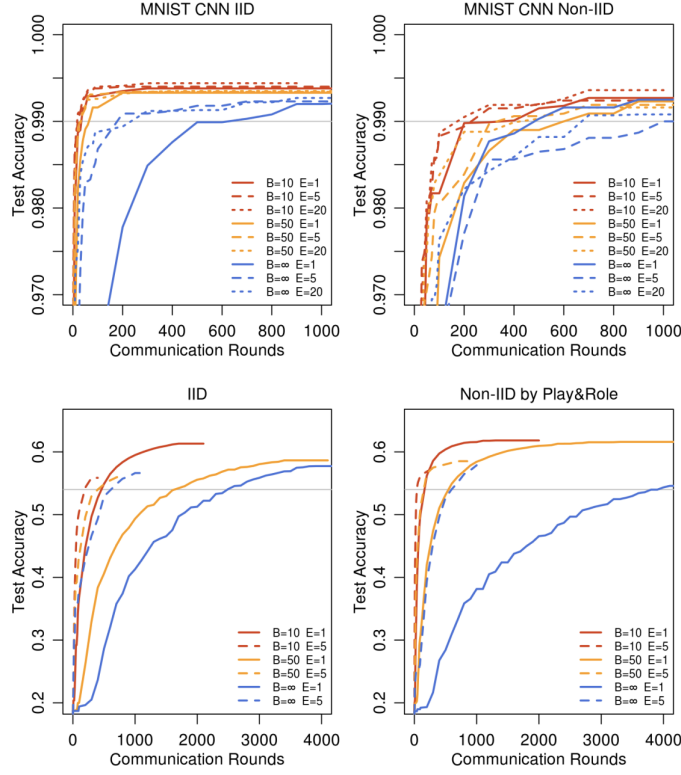


Figura 2.1: Accuratezza di test di FedAvg al variare di B ed E [15].

Il Federated Averaging si può riassumere in un algoritmo.

Algorithm 1 Federated Averaging lato server

```

1: Inizializza  $w_0$ 
2: for each round  $t = 1, 2, \dots$  do
3:    $S_t \leftarrow$  random set clients
4:   for each client  $k \in S_t$  in parallel do
5:      $w_{t+1}^k \leftarrow \text{ClientUpdate}(k, w_t)$ 
6:   end for
7:    $w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$ 
8: end for
9:
10: function CLIENTUPDATE( $k, w$ )
11:   Dividi il dataset in batch di grandezza  $B$ 
12:   for each epoch  $i = 1, \dots, E$  do
13:     for each batch  $b \in B$  do
14:        $w \leftarrow w - \eta \nabla \ell(w; b)$ 
15:     end for
16:   end for
17:   return  $w$ 
18: end function

```

Oltre al FedAvg, negli anni sono stati proposti svariati algoritmi, ciascuno con l'intento di ottimizzare un certo aspetto del federated learning. La funzione di aggregazione

utilizzata all'interno del Federated Averaging potrebbe infatti presentare limiti significativi in alcuni contesti. In scenari non-IID, i pesi di due neuroni appartenenti allo stesso layer, ma addestrati su dataset diversi, potrebbero rappresentare feature completamente diverse. Una media diretta di questi rischia dunque di perdere entrambe le informazioni e produrre un modello globale non funzionante in determinate circostanze. Un paradigma proposto per superare questo problema è quello del feature-level aggregation, in cui, invece di considerare i pesi dei neuroni, l'attenzione si sposta sulle feature apprese da questi. L'idea alla base è quella di confrontare le attivazioni dei neuroni quando elaborano i dati, ossia se producono output simili sugli stessi input, comprendendo in questo modo quali neuroni rappresentano la stessa informazione. Tecnicamente viene svolta una group convolution dei neuroni, raggruppando tra loro quelli che hanno imparato la stessa feature, e successivamente si calcola la media di ciascun gruppo, evitando di mischiare tra loro neuroni eterogenei.

Altri approcci cercano di mitigare i problemi relativi al federated learning discussi in precedenza. Una prima sfida è quella dell'aggregazione di modelli generati da ambienti eterogenei, in cui l'eterogeneità può essere a livello di dati, modelli o sistemi hardware. La presenza di dati eterogenei può essere risolta con alcune strategie di apprendimento già analizzate in precedenza, come il meta-learning o il transfer learning. Un'altra tecnica è quella basata sul clustering, che, invece di addestrare un modello unico, ne addestra diversi, uno per ogni gruppo di client con dati simili tra loro. In questo modo si ottengono modelli altamente specifici e personalizzati per ciascun tipo di client. Per affrontare il problema dell'eterogeneità dei modelli, in cui client distinti hanno bisogno di modelli differenti, è di grande utilità il framework HeteroFL [21]. Qui il server mantiene un modello centrale grande e generale, mentre a ciascun client viene assegnato un modello di dimensioni più contenute, in funzione delle proprie necessità. Durante la fase di aggregazione vengono dunque aggiornati solamente i parametri corrispondenti alla parte di rete addestrata dai modelli più piccoli.

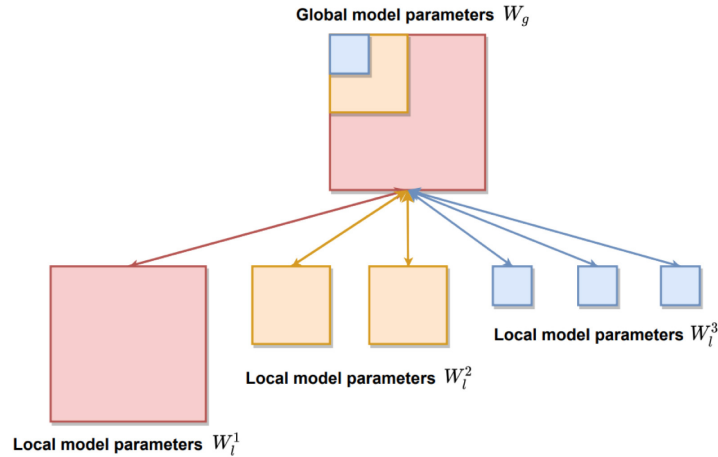


Figura 2.2: HeteroFL [22].

Infine, l'eterogeneità dei sistemi, che spesso porta a un rallentamento generale dell'apprendimento a causa dei dispositivi meno potenti, può essere risolta tramite algoritmi di selezione dei client intelligenti, come Oort [23] e PyramidFL [24]. Questi consentono al sistema di federated learning di coinvolgere nell'addestramento solo i

client più utili, senza dover attenderli tutti. Ciò viene fatto analizzando la qualità dei dati dei client, la loro velocità e affidabilità, svolgendo una selezione ad ogni round.

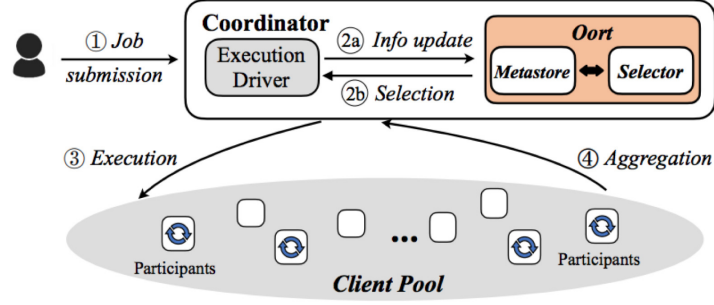


Figura 2.3: Oort [22].

Un altro problema da affrontare è quello della sicurezza, molto importante nella maggior parte dei contesti di applicazione del federated learning e minacciata da svariate tipologie di attacchi. Un esempio è quello dei backdoor, che consistono nella manipolazione del modello globale in modo tale che questo risponda in un certo modo a specifici input, senza alterare le prestazioni su dati generici. Ciò può essere svolto inserendo esempi malevoli nei dataset dei dispositivi edge, cercando di evitare che il processo di aggregazione dei modelli ne diluisca l'effetto. Attacchi ai gradienti vengono invece usati per ottenere informazioni sensibili sui dati di addestramento partendo dai gradienti condivisi dai client in fase di aggiornamento. Per difendersi da queste minacce possono essere implementate diverse tecniche, tra cui quella della differential privacy [25] è tra le più utilizzate. Il principio alla base è quello dell'aggiunta di rumore all'interno dei parametri aggiornati del modello, cercando di non compromettere significativamente le prestazioni. Un'altra linea di difesa consiste nell'uso dell'Homomorphic Encryption [26], con la quale il server è in grado di aggregare i modelli dei client lavorando su dati criptati. Il sistema BatchCrypt [27], ad esempio, prevede cinque passaggi principali: generazione e distribuzione della chiave di cifratura, aggiornamento locale dei gradienti, cifratura dei gradienti, aggregazione sul server e decifratura da parte dei client per il training successivo.

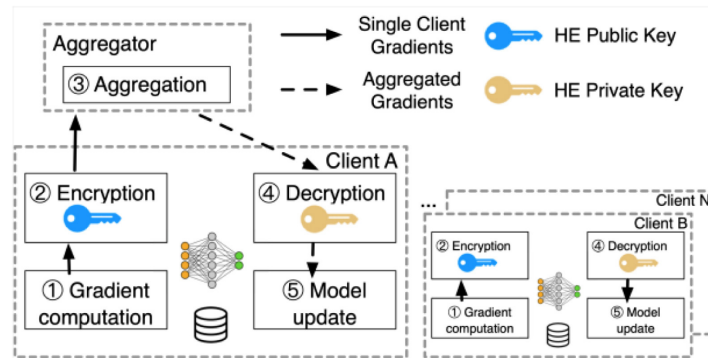


Figura 2.4: BatchCrypt [22].

Infine, l'utilizzo di un Trusted Execution Environment [28] consente di migliorare la sicurezza utilizzando un approccio di tipo hardware. Un TEE è infatti una zona isolata nel processore principale, in cui viene garantita la confidenzialità e l'integrità

dei dati e dei programmi tramite un insieme di tecniche hardware, firmware e l'utilizzo di istruzioni speciali. Una difesa di questo tipo è più efficiente in termini computazionali rispetto alle HE e DP discusse in precedenza. I TEE vengono quindi usati nei dispositivi locali per svolgere gli addestramenti, come accade nel framework PPFL [29], in cui viene adottato un training greedy layer-wise per superare i limiti di memoria. Questa strategia addestra uno o più layer alla volta, invece di aggiornare simultaneamente tutti i parametri del modello.

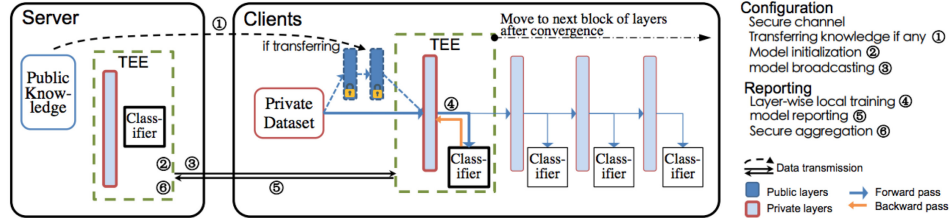


Figura 2.5: PPFL [22].

Il Federated Learning è attualmente un campo di ricerca molto attivo e, nonostante siano stati fatti notevoli progressi negli ultimi anni, ci sono ancora diverse sfide da affrontare. Un importante contesto di studio è quello della diagnostica dell'eterogeneità di un sistema FL. Attualmente ci sono diverse metriche che consentono di stimare la diversità a livello di dati e sistemi, tuttavia queste non sono facili da calcolare prima di iniziare l'addestramento. È quindi fondamentale sviluppare metodi rapidi per valutare il livello di eterogeneità, in modo tale da comprendere quali tecniche utilizzare e come ottimizzarle. Un'altra utile direzione di ricerca è quella dell'utilizzo di altre metodologie di apprendimento per il federated learning. Finora gli studi si sono principalmente focalizzati sull'utilizzo di apprendimento supervisionato, assumendo la presenza di etichette su ciascun dato. Tuttavia, in molti casi d'uso reali, è ampia la presenza di dati non etichettati sui dispositivi terminali. Inoltre, in determinati sistemi, può essere necessario l'utilizzo di metodologie dedicate, come l'analisi esplorativa o l'apprendimento per rinforzo. Affrontare questi scenari rappresenta una sfida significativa, in quanto richiederà l'adozione di nuove metodologie per risolvere i problemi già discussi sulla scalabilità, eterogeneità e privacy. Infine, essendo un campo relativamente giovane, il federated learning non possiede benchmark realistici che rappresentano uno standard. Ciascun gruppo di ricerca sull'argomento utilizza dataset, protocolli e algoritmi distinti, rendendo complicato il confronto tra le varie soluzioni. Delle metriche ampiamente accettate e riproducibili consentirebbero al federated learning di progredire in maniera più pratica e veloce.

2.2 Il framework Flower

Il framework Flower [30] è una piattaforma open source che semplifica la creazione, progettazione e testing di sistemi di federated learning. Prima della sua ideazione, un grande limite nel progresso tecnologico del FL era dato dal fatto che vi era una forte disparità tra la ricerca e la produzione in questo ambito. Il federated learning è caratterizzato infatti da sistemi solitamente molto complessi per via dell'eterogeneità dei dispositivi, della banda utilizzata e delle diverse tipologie di algoritmi di aggregazione.

esistenti. Per questo motivo difficili erano gli esperimenti in materia, dato che i pochi framework disponibili presentavano limiti di scalabilità e prevedevano principalmente la messa alla prova dei diversi algoritmi di federated learning, senza supporto all'esecuzione su dispositivi edge. Per questo, mentre a livello di produzione venivano utilizzati sistemi client nell'ordine di migliaia o milioni, gli articoli scientifici svolgevano prove su popolazioni di 100 client o poco più, risultando così poco rappresentativi. Flower, in questo contesto, è stato sviluppato con l'intento di avere un framework in grado di eseguire esperimenti con dispositivi numerosi ed eterogenei, che fungesse da fattore comune per le ricerche in materia. Tale approccio avrebbe reso possibili diversi casi d'uso che fino ad allora non erano esplorabili in ambito di sperimentazione, come esperimenti su larga scala, la possibilità di simulare e utilizzare dispositivi eterogenei, permettere una transizione semplice e fluida da un ambiente di simulazione a uno di esecuzione e consentire di utilizzare diversi framework di machine learning all'interno dello stesso carico di lavoro. Per questo la realizzazione di Flower è stata basata sui seguenti obiettivi di design.

- Scalabile. Flower deve poter scalare a un numero elevato di clienti, così da simulare contesti realistici.
- Client-agnostico. Flower non deve dipendere da una singola tipologia di client, ma deve poter essere interoperabile con diversi linguaggi di programmazione, sistemi operativi e hardware.
- Communication-agnostico. Flower deve prevedere l'utilizzo di differenti metodi di comunicazione tra i nodi.
- Privacy-agnostico. Flower deve adattarsi ai requisiti di privacy dei vari scenari di FL, che sono potenzialmente differenti. Per questo deve fornire supporto alle principali tecniche di cybersecurity esistenti in ambito.
- Flessibile. Flower deve ridurre al minimo il carico di lavoro necessario per implementare nuove tipologie di ricerche e l'adozione di approcci innovativi, vista la rapida evoluzione del federated learning.

L'architettura del core framework di Flower si basa sul federated learning come processo che utilizza due tipologie di computazione: globale e locale. Le computazioni globali vengono eseguite lato server e hanno il compito di gestire e coordinare l'intero processo di apprendimento. Le computazioni locali, invece, vengono eseguite dai singoli client e sono le uniche ad avere accesso ai dati locali di addestramento. Flower considera dunque due livelli per riflettere questa distinzione, permettendo agli sviluppatori di utilizzare un'architettura flessibile.

- Il livello globale gestisce tutta la logica del sistema e comprende operazioni come la selezione dei client che partecipano al round di addestramento, la configurazione dei round, l'algoritmo di aggregazione e l'eventuale valutazione centralizzata del modello aggiornato. Per comprendere tutto ciò l'architettura si compone di tre componenti principali:

1. ClientManager. Gestisce l'insieme dei client connessi al server, rappresentati come oggetti ClientProxy. Ciascun ClientProxy invia e riceve messaggi

utilizzando il protocollo di Flower, il quale definisce uno standard per la comunicazione tra server e client, garantendo flessibilità.

2. **Strategy.** Contiene al suo interno tutta la logica dello specifico algoritmo di federated learning utilizzato. È un'interfaccia astratta a cui vengono delegate le decisioni su come selezionare i client, come inizializzare i pesi e come aggregare i risultati. Flower fornisce anche implementazioni pronte all'uso per algoritmi noti in letteratura, come il già citato FedAvg.
 3. **FL Loop.** Costituisce il ciclo principale di esecuzione di una sezione di federated learning. Non contiene una logica di apprendimento specifica, ma chiede alla Strategy come configurare il round. Successivamente invia le configurazioni ai client, attende e passa le risposte alla Strategy per svolgere l'aggregazione.
- Il livello locale si occupa dell'addestramento dei modelli presenti nei client e della valutazione di questi sui dati locali. Tramite l'astrazione Client è possibile implementare solo le funzioni di addestramento e valutazione, mentre la gestione della comunicazione e della sincronizzazione viene svolta da Flower. In questo modo il framework risulta anche agnostico rispetto al framework di machine learning utilizzato localmente, dunque è possibile integrare client basati su strumenti diversi, come PyTorch e TensorFlow, anche in combinazione sullo stesso dispositivo.

L'architettura è stata realizzata appositamente in modo tale che il server non conosca la natura dei client connessi, consentendo eterogeneità nell'addestramento. Il framework gestisce dunque tutte le complessità di livello più basso, come la gestione delle connessioni e il ciclo di vita.

All'interno di Flower è integrato il Virtual Client Engine (VCE), un componente che permette di ottimizzare l'uso delle risorse hardware per svolgere addestramenti con un elevato numero di client. Il VCE consente di svolgere round su larga scala anche quando l'hardware utilizzato presenta dei limiti, e lo fa tramite tecniche di virtualizzazione e simulazione dei client. Questi vengono dunque astratti a entità virtuali, e il VCE ne gestisce l'intero ciclo di vita cercando di massimizzare il grado di ottimizzazione e parallelizzazione per svolgere un uso efficiente delle risorse. Il funzionamento del VCE si basa su un'esecuzione dei client in maniera resource-aware, tenendo conto di alcuni parametri fondamentali. Questi sono i vincoli di risorse di ciascun client e gli iperparametri dell'algoritmo di FL, come il numero di client. Il VCE consente in questo modo di eseguire lo stesso esperimento indipendentemente dall'ambiente di esecuzione, potendo utilizzare anche dispositivi caratterizzati da poche risorse.

Per far funzionare correttamente l'addestramento, Flower avrebbe bisogno di eseguire un'istanza completa dei codici presenti nei client, che si occupano sia del training che della logica di comunicazione con il server. Tuttavia, molti dispositivi edge reali, soprattutto quelli di tipo commodity come smartphone e tablet, spesso operano con stack software rigidi (a volte proprietari) e limitati, rendendo difficile l'esecuzione diretta di modelli di Machine Learning. Per ovviare a questo problema, Flower implementa l'Edge Client Engine, un componente che permette di suddividere il client in due parti, separando la logica di addestramento da quella di comunicazione. In questa maniera ciascun nodo ha solo bisogno di implementare un modulo leggero che si occupa di in-

viare e ricevere messaggi secondo il Flower Protocol, mentre l'effettivo addestramento viene delegato a un software personalizzato per il suo ambiente nativo.

Per quanto riguarda la sicurezza nell'invio dei parametri dei modelli aggiornati, Flower fornisce un'implementazione integrata dei protocolli di SecAgg [31] e SecAgg+ [32]. Questi seguono il seguente meccanismo:

- Ogni client utilizza delle chiavi crittografiche non condivise con il server per mascherare i propri aggiornamenti.
- Il server riceve gli aggiornamenti che non sono comprensibili singolarmente, ma hanno una proprietà tale per cui la somma complessiva delle maschere si annulla durante l'aggregazione. In questo modo il server otterrà soltanto il modello finale aggregato, senza poter osservare i singoli contributi.

Questo protocollo è indipendente da qualsiasi hardware o framework ML, garantendo compatibilità e flessibilità, e comporta un ridotto overhead computazionale.

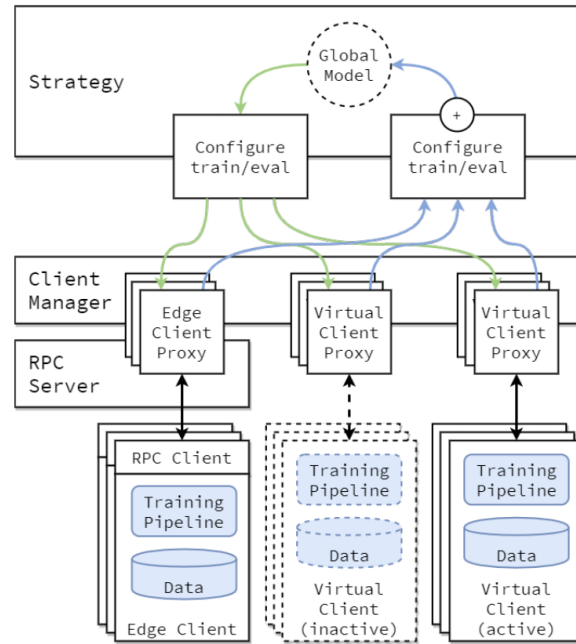


Figura 2.6: Architetture del core framework di Flower [30].

Per quanto riguarda l'implementazione tratteremo solo alcune parti dell'intera architettura. Un aspetto fondamentale per il corretto funzionamento di sistemi di FL è quello dello stack di comunicazione tra il server e i numerosi client. Flower, per questa finalità, implementa il proprio protocollo di comunicazione basandosi su un'infrastruttura di stream bidirezionali gRPC, una tecnologia di comunicazione sviluppata da Google. Qui vengono definiti i diversi tipi di messaggi che possono essere scambiati tra client e server: parametri di inizializzazione del modello, pesi aggiornati, risultati delle valutazioni locali e notifiche di errore o completamento. La scelta di gRPC risiede nelle necessità di avere una comunicazione stabile, efficiente e scalabile, spesso complicata in un ambiente edge. gRPC implementa infatti una serializzazione binaria efficiente dei messaggi tramite Protocol Buffers, riducendo al minimo la loro dimensione. In questo modo è possibile evitare colli di bottiglia causati da overhead di comunicazione in

scenari a bassa larghezza di banda, come nel caso dei dispositivi mobili. Inoltre, questo protocollo consente di scambiare più messaggi in maniera consecutiva senza dover ri-stabilire la connessione, mantenendo un flusso continuo di dati tra client e server e migliorando la latenza complessiva. Un altro vantaggio è dato dall'utilizzo di Protocol Buffers, con i quali è possibile generare in maniera automatica implementazioni del protocollo di comunicazione in diversi linguaggi di programmazione, garantendo compatibilità multi-linguaggio. Questo consente di rispettare il requisito sull'eterogeneità, dato che è possibile sviluppare client federati su piattaforme e sistemi operativi di diverso tipo. Un singolo flusso di comunicazione tra server e client avviene nel seguente modo:

1. Ricezione: il client riceve dal server messaggi in array di byte.
2. Deserializzazione: i messaggi binari vengono tradotti in istruzioni utilizzabili.
3. Esecuzione: il client esegue le singole istruzioni, ad esempio addestrando il modello locale.
4. Serializzazione dei risultati: i risultati dell'addestramento vengono serializzati in messaggi binari.
5. Invio al server: i byte appena ottenuti vengono inviati al server per l'elaborazione.

L'utente ha anche la possibilità di scegliere un proprio metodo di serializzazione personalizzato, ad esempio al fine di comprimere la dimensione o criptare i dati per una maggiore sicurezza.

Nonostante Flower usi gRPC come stack di comunicazione, l'architettura modulare del server consente alle componenti interne di essere indipendenti da esso. È possibile quindi sostituire gRPC con altri framework RPC senza dover riscrivere tutto il server, garantendo flessibilità. Come accennato in precedenza, ciascun client disponibile per il server ai fini dell'addestramento è rappresentato da un oggetto di tipo ClientProxy. ClientProxy è definita come un'interfaccia, in questo modo è possibile implementare il meccanismo di comunicazione in maniera differente per ciascun client.

Un ultimo importante aspetto è quello del Virtual Client Engine. Come accennato in precedenza, questo componente permette di eseguire esperimenti di FL su larga scala anche su hardware limitato. Per fare ciò viene creato un oggetto ClientProxy per ciascun client simulato, ma la sua inizializzazione avviene solamente nel momento in cui ci sono le risorse necessarie per eseguire il relativo task. Il VCE si basa sul framework Ray [33], il quale può decidere se eseguire i task lato client in maniera sequenziale o parallela, così da adattarsi alle risorse disponibili.

Per quanto riguarda la valutazione del framework Flower, al momento del suo lancio sono stati condotti alcuni esperimenti per mettere alla prova diversi aspetti, tra cui la scalabilità, l'eterogeneità e la privacy. Per quanto riguarda il primo punto, è stato preso in considerazione il concetto secondo cui nel federated learning, coinvolgere un maggior numero di client nei vari round di addestramento non comporta necessariamente un minor tempo di convergenza. È stato osservato che spesso vi è una soglia empirica oltre la quale aumentare il numero di client comporta un rallentamento, dato che il lavoro svolto dagli algoritmi di aggregazione viene complicato da dati locali troppo eterogenei. Questo fenomeno è stato studiato in Flower svolgendo un esperimento su

larga scala sulla personalizzazione di un modello pre-addestrato, DistilBERT [34], che si occupa della previsione del punteggio assegnato da un utente a un libro prendendo in input il testo della recensione. È stato usato un insieme di test costituito da 1 milione di client, dal quale ne viene scelto un numero fisso per ogni round. I risultati hanno dimostrato che per un numero di client tra 10 e 500, l'aumento dei partecipanti comporta un'accelerazione nella convergenza, mentre superata questa soglia si nota un rallentamento. Con questo esperimento è stato possibile provare che Flower è adatto a condurre esperimenti su larga scala.

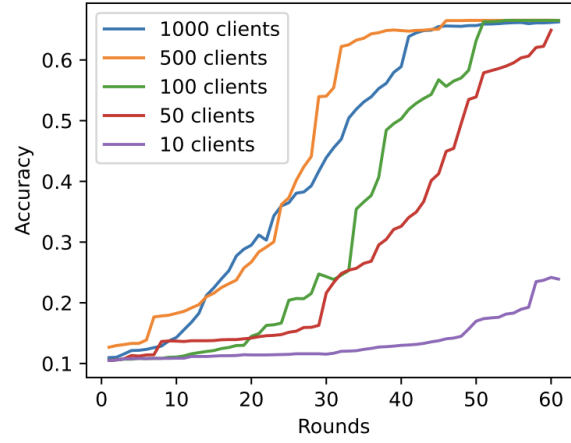


Figura 2.7: Esperimento su larga scala di Flower con DistilBERT [30].

Altri esperimenti hanno confrontato le performance di Flower con quelle di altri framework FL noti, al variare di parametri come il numero di client attivi per round (c) e il numero di epoche locali di addestramento (l). I risultati dimostrano come Flower sia uno tra i più adatti alla ricerca, dato che, pur avendo un leggero overhead su parametri piccoli, la sua architettura scalabile è tra le migliori negli scenari più realistici.

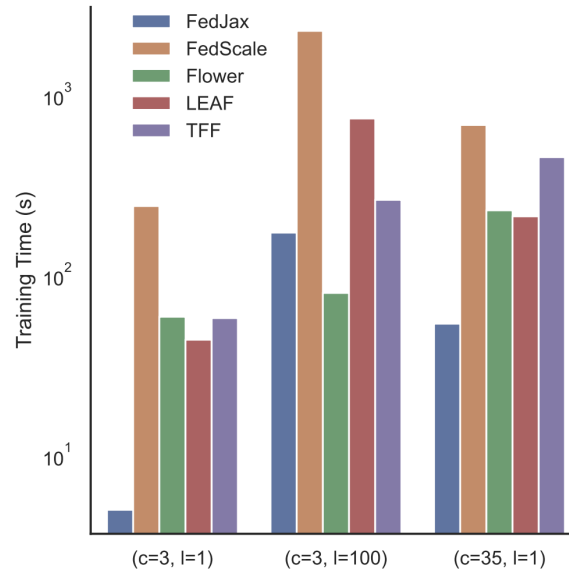


Figura 2.8: Confronto delle performance di Flower con quelle di altri framework [30].

Per quanto riguarda l'eterogeneità, altri esperimenti hanno dimostrato come Flower sia in grado di simulare scenari reali tramite la possibilità di definire il tempo di esecuzione e la velocità di rete di ogni specifico client. In questo modo è possibile comprendere come le performance del sistema variano in funzione della selezione dei client e del tempo limite di completamento dei round, comprendendo dispositivi con capacità eterogenee e distribuiti geograficamente.

Infine, sono state condotte simulazioni per misurare l'impatto sull'overhead computazionale del server e sul traffico di comunicazione dato dall'implementazione della secure aggregation. È stato dimostrato che i tempi di esecuzione e il traffico aumentano linearmente con la dimensione del vettore inviato tra client e server. Questo conferma che Flower è estremamente efficace nell'implementazione della secure aggregation, dato che SecAgg e SecAgg+ presentano di default complessità lineare rispetto alle dimensioni.

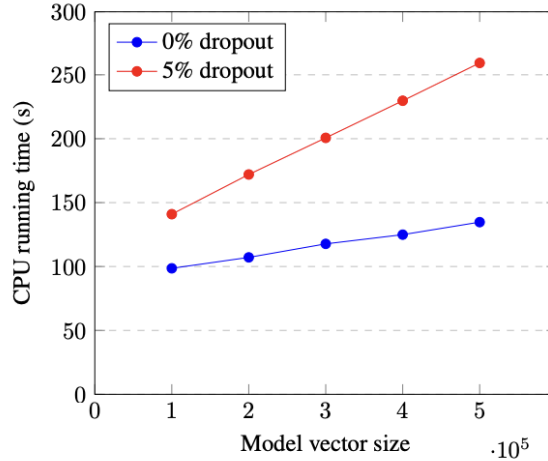


Figura 2.9: Performance della secure aggregation in Flower [30].

2.3 Applicazioni e casi d'uso del federated learning

In questa sezione vengono presentati i principali campi di applicazione e casi d'uso del federated learning, così da poter comprendere più a fondo il potenziale di questo paradigma di apprendimento.

Un importante ambito è quello della cybersecurity, in particolare il rilevamento di anomalie. Con un sistema di federated learning è infatti possibile identificare comportamenti atipici di dispositivi all'interno di una rete, che sono solitamente causati da compromissioni o attività sospette. Un esempio è il progetto DIoT [35], il quale raggiunge l'obiettivo di un modello globale capace di riconoscere anomalie su molti tipi di dispositivi e comportamenti diversi. Il punto di forza di tale sistema è il fatto che opera in maniera completamente autonoma, non richiedendo intervento umano per etichettare i dati e permettendo così di utilizzare grandi flussi di informazioni impossibili da gestire in un contesto centralizzato. DIoT presenta un'architettura composta da due componenti, il Security Gateway e il Servizio di Sicurezza IoT, i quali gestiscono rispettivamente il lavoro locale e quello del server. Il primo esegue infatti all'interno dei singoli dispositivi, ed è responsabile sia dell'identificazione di questi quando vengono aggiunti alla rete, sia del rilevamento delle anomalie tramite il riconoscimento di pat-

tern di comportamento. Il Servizio di Sicurezza IoT si occupa invece del mantenimento dei modelli, la distribuzione di questi e l'aggregazione finale. I risultati sperimentali di questo progetto mostrano un tasso di rilevamento del 95% e un'assenza di falsi positivi, confermando l'utilità del federated learning in un tale contesto. Grazie a questo, infatti, viene garantita la privacy dei dati provenienti dai dispositivi, ottenendo scalabilità, grazie al funzionamento autonomo, e adattabilità, dato che i modelli locali possono essere aggiornati continuamente imparando così a gestire attacchi emergenti o sconosciuti.

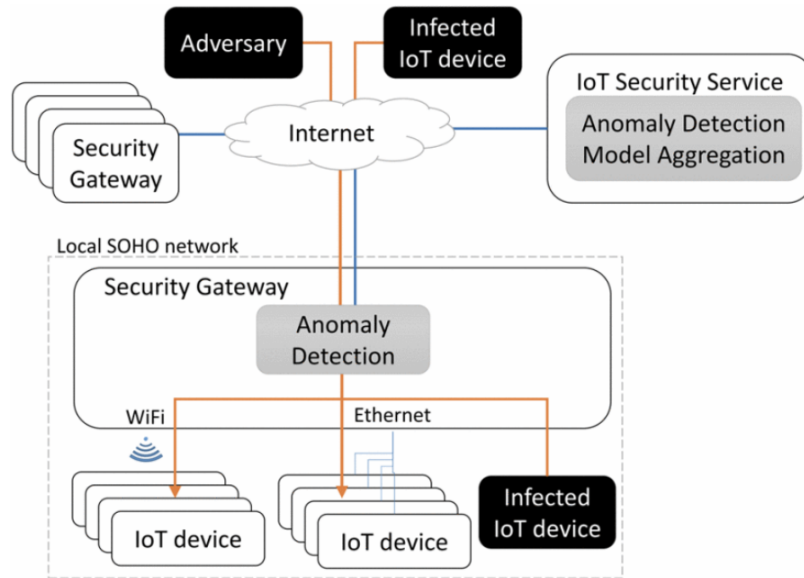


Figura 2.10: DIoT [36].

Quello dei trasporti è un ulteriore campo di applicazione del federated learning. Qui infatti sussistono numerosi veicoli che contengono un numero sempre più elevato di sensori in grado di rilevare importanti informazioni su posizione, velocità, condizione della strada e altri dati utili. In questo contesto l'utilizzo di modelli addestrati in maniera centralizzata presenta diverse limitazioni, in quanto questi non riescono ad adattarsi alla natura dinamica e in continua evoluzione degli scenari stradali. Anche per quanto riguarda la predizione dei flussi di traffico tale paradigma fallisce, in quanto tecniche di analisi di questo tipo hanno bisogno di grandi quantità di dati provenienti da località geografiche differenti.

L'ambito sanitario è uno dei più adatti al federated learning. Questo è infatti caratterizzato da dati clinici frammentati, spesso caratterizzati da un'elevata sensibilità e molte soluzioni in questo campo richiedono un'enorme quantità di dati distribuiti in più strutture sanitarie. Un primo esempio di applicazione riguarda l'utilizzo di sistemi di machine learning per analizzare le cartelle cliniche elettroniche, potendo così prevedere l'insorgenza di malattie e analizzare altri eventi clinici. L'utilizzo di un modello addestrato in maniera centralizzata in questo contesto sarebbe molto complicato, dato che i dati clinici sono contenuti in ospedali e cliniche diverse, e il loro trasferimento è solitamente protetto da vincoli di privacy. Molte soluzioni proposte per questo problema si basano su algoritmi di federated learning, come il Community Based Federated Learning [37], che contiene dati relativi a oltre 200.000 pazienti ricoverati in 200 ospedali. Il CBFL si basa su tre fasi principali.

1. Addestramento di encoder locali in grado di comprimere, in ogni clinica, i dati clinici dei pazienti.
2. Invio dei dati ottenuti al server centrale, il quale esegue un clustering per raggruppare i pazienti in comunità con caratteristiche simili.
3. Addestramento in maniera federata di più modelli, ciascuno per ogni comunità trovata nella fase precedente. In questo modo si ottiene un modello personalizzato per ogni tipo di paziente, garantendo una maggiore accuratezza.

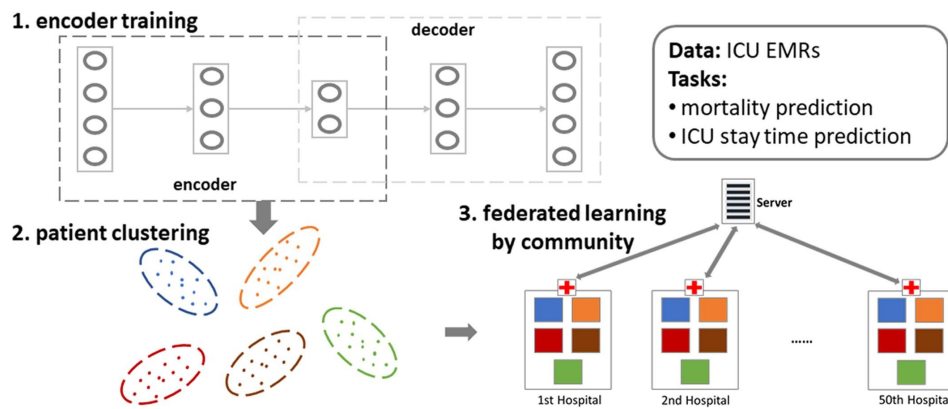


Figura 2.11: Community Based Federated Learning [37].

Un altro caso di successo è quello della segmentazione dei tumori cerebrali a partire da risonanze magnetiche. Anche qui, data la scarsità di dati esistenti per patologie rare, l'approccio federato è fondamentale per ottenere un buon modello senza richiedere la condivisione diretta di dati sensibili. Soluzioni simili sono state implementate nell'ambito della Named-entity recognition, ovvero l'identificazione di nomi di farmaci, reazioni avverse e sintomi partendo da testi medici non strutturati. Anche qui la grande quantità di dati etichettati richiesti suggerisce l'uso di tecniche di federated learning, che in molti progetti hanno superato le prestazioni dei modelli tradizionali.

Infine, un ulteriore settore in cui possono essere applicati i vantaggi del federated learning è quello della computer vision. Qui si trova un'ampia varietà di contesti che necessitano la raccolta di grandi quantità di dati visivi, spesso di natura sensibile, come il monitoraggio della sicurezza e il riconoscimento facciale. Componenti come visual object detectors, ad esempio nel monitoraggio di pericoli d'incendio o nella sicurezza industriale, hanno bisogno di raccogliere dati provenienti da telecamere distribuite geograficamente, e la raccolta centralizzata di questi comporta problemi in termini di costo e perdita di controllo da parte degli utenti. La piattaforma FedVision [38], che consente di semplificare l'uso del Federated Learning nei progetti di computer vision, è largamente utilizzata da grandi aziende per applicazioni di sicurezza basate su computer vision.

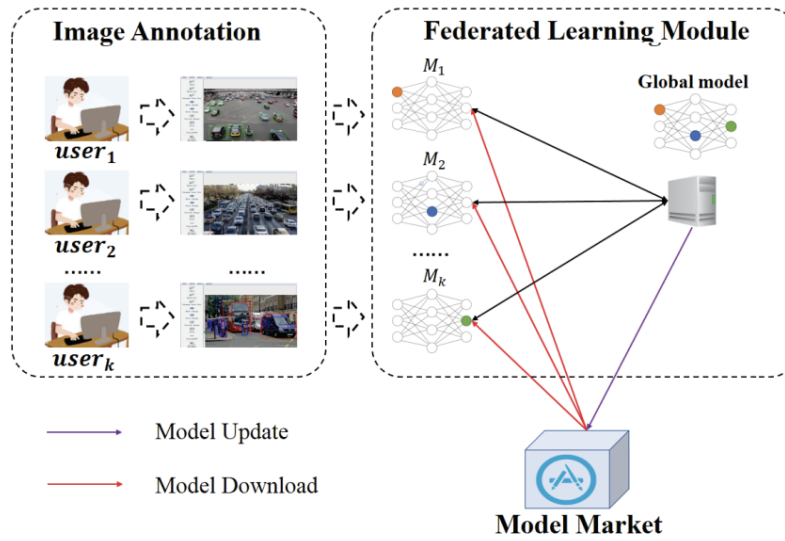


Figura 2.12: FedVision [38].

Applicazioni di Augmented Reality hanno a loro volta bisogno di processare enormi quantità di dati visivi e devono farlo in tempo reale. I severi vincoli di latenza che spesso caratterizzano queste applicazioni rendono impraticabile il continuo invio di dati ad un server centrale, anche per i rischi di violazione della privacy.

Capitolo 3

Implementazione di un sistema di riconoscimento facciale con Federated Learning

3.1 Obiettivi sperimentali

Al fine di mettere alla prova le conoscenze teoriche apprese e l'effettiva utilità del federated learning, è stato sviluppato un progetto sperimentale per l'addestramento federato di un modello di machine learning. Gli obiettivi di un tale studio consistono nel dimostrare la correttezza tecnica dell'approccio, tramite la valutazione di metriche qualitative, e la sua fattibilità in scenari reali, esaminandone prestazioni ed eventuali limiti.

Per fare ciò è stato deciso di impostare il progetto su un caso d'uso diffuso e capace di trarre vantaggio dall'utilizzo su dispositivi edge con questo paradigma di apprendimento. Tra i diversi contesti che rispettavano queste specifiche, la scelta è ricaduta sul riconoscimento facciale, tecnologia sempre più utilizzata e che presenta numerose sfide. L'identificazione tramite il volto trova molte applicazioni in contesti edge, dato che sistemi di accesso a strutture, smartphone e altri dispositivi IoT ne richiedono un sempre maggiore utilizzo. In tali situazioni la disponibilità di connettività con un server centrale non è sempre garantita, e le risorse computazionali sono spesso limitate se si opera su un singolo dispositivo. La natura dei dati utili all'addestramento di un sistema di questo tipo è un ulteriore importante ostacolo nell'adozione di tecniche centralizzate. Questi vengono infatti generati dai sensori presenti sui dispositivi terminali, e ciò comporta una loro distribuzione geografica non indifferente. Inoltre, le immagini facciali rappresentano dati biometrici altamente sensibili e difficili da anonimizzare, rendendo pericoloso e complicato il loro invio verso data center. Il federated learning permette di superare questi limiti adattando il processo di addestramento alle condizioni dettate dal contesto di applicazione, consentendo di distribuire il carico di lavoro, ridurre la dipendenza da un'infrastruttura centralizzata e garantire una maggiore privacy.

L'obiettivo del progetto è quello di dimostrare la fattibilità di un tale approccio di apprendimento tramite la progettazione e l'implementazione di un sistema simulato. Come visto in precedenza infatti, la simulazione in un tale ambito è necessaria per ambienti di ricerca e prototipazione, caratterizzati solitamente da un numero non

sufficiente di client. Il sistema proposto sarà in grado di allenare un modello di deep learning adatto al riconoscimento facciale non pre-addestrato, che presenta quindi pesi non adatti ad estrarre embeddings precisi dei volti forniti in input. Lo sviluppo di algoritmi in grado di calcolare metriche di valutazione relative al modello ci permetterà di valutare le sue performance prima e dopo l'addestramento, giudicandone l'evoluzione prestazionale. Le metriche adottate per l'analisi sono la Top-1 Accuracy e Top-5 Accuracy. Questa scelta è dettata dalla decisione di trattare un contesto di classificazione, in cui il modello può distinguere identità diverse all'interno di un ampio set. In altre parole, fornita in input l'immagine di un volto, tramite l'estrazione di embeddings accurati il sistema deve restituire la label della persona più simile tra tutte quelle presenti nel dataset. La Top-1 Accuracy rappresenta la metrica più diretta per valutare ciò. Essa coincide con la percentuale dei casi in cui l'etichetta predetta come più probabile dal modello coincide con quella corretta. Formalmente:

$$\text{Top-1 Accuracy} = \frac{\text{n. previsioni corrette}}{\text{tot. campioni}} \quad (3.1)$$

Siccome i dataset utilizzati per il face recognition contengono solitamente un numero elevato di identità differenti, la Top-1 Accuracy potrebbe essere penalizzante nel comprendere il corretto andamento dell'addestramento. In una tale metrica infatti, se la classe corretta non è al primo posto ma vicinissima, l'intera predizione viene considerata errata. Per questo motivo analizziamo anche la Top-5 Accuracy, che allarga lo spettro di valutazione. Qui infatti la previsione è considerata corretta se la classe corretta compare tra le prime cinque predette dal modello.

$$\text{Top-5 Accuracy} = \frac{\text{n. previsioni con classe corretta tra le prime 5}}{\text{tot. campioni}} \quad (3.2)$$

In questo modo viene misurata la capacità del modello di restringere le possibilità in uno spazio ristretto. Nell'ambito della valutazione di un modello di riconoscimento facciale sussistono altre metriche frequentemente utilizzate in letteratura. Un esempio è quello della Precision, che misura la proporzione di previsioni positive corrette tra tutte quelle predette come positive. La ROC curve mostra invece il trade-off tra True Positive Rate e False Positive Rate al variare di una soglia di decisione. Tali misure non sono però compatibili con il nostro ambito di ricerca, in quanto trattano la biometria in modalità verification (1:1 matching), ovvero quando, dati due volti, si vuole sapere se appartengono alla stessa persona.

Il progetto non si limita alla sola valutazione del processo di addestramento, ma mira a fornire un'implementazione modulare e riutilizzabile nel campo, in grado di poter essere utilizzata per progetti futuri con più risorse computazionali a disposizione.

3.2 Architettura del sistema

Durante la prima fase di progetto è stata delineata l'infrastruttura generale del sistema con l'obiettivo di garantire scalabilità e facilità di manutenzione. La scelta del framework utilizzato per la simulazione dell'addestramento è risultata determinante per alcune scelte architetturali, dato che Flower, selezionato per i numerosi vantaggi analizzati in precedenza, impone una struttura ben definita. Il modello adottato è di tipo

client-server, in cui occorre definire la struttura del server centrale, dei client e della strategia di federated learning adottata.

Tutto il sistema è stato diviso nei seguenti moduli:

- Modulo server. Qui viene definita e creata l'istanza di **ServerApp**, alla quale viene fornito un oggetto di tipo **ServerAppComponents**, che mantiene al suo interno tutto il funzionamento del server centrale. Questo, delegando alcune scelte alle Strategy utilizzata, coordina tutto il processo di federated learning. Si occupa infatti di inizializzare i pesi del modello globale, distribuirli ai client ad ogni round, raccogliere gli aggiornamenti locali e terminare con l'aggregazione.
- Modulo client. Definisce e crea l'istanza di **ClientApp**, che genera un **FlowerClient** per ciascun dispositivo che contribuisce all'addestramento federato. Qui vengono definite le funzioni fit ed evaluate, adatte rispettivamente al training e alla valutazione locale del modello. Il ciclo di vita di un singolo client inizia con la ricezione dei pesi globali dal server, passa per le fasi di allenamento e testing locale, e termina con la restituzione al server dei nuovi parametri e delle metriche ottenute.
- Modulo di testing. Utile al calcolo delle metriche atte alla valutazione del modello (Top-1 Accuracy e Top-5 Accuracy) prima e dopo l'addestramento.

La struttura modulare di Flower ha permesso inoltre di adottare un approccio a livelli, facendo comunicare i diversi moduli tramite interfacce ben definite, come accade con l'utilizzo di array numerici per la comunicazione dei pesi del modello. In questo modo ciascun attore non è influenzato dalle altre implementazioni, permettendo al server di non conoscere i dettagli del training locale e garantendo che i client rimangano isolati dalla logica di aggregazione.

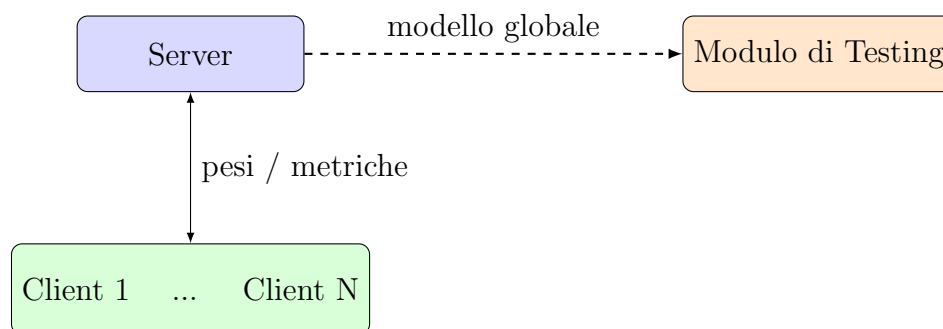


Figura 3.1: Schema riassuntivo dell'architettura di sistema.

3.3 Implementazione del sistema

In questa sezione vengono analizzate in dettaglio le scelte implementative e visionate parti di codice utili alla comprensione del progetto.

Per quanto riguarda il linguaggio di programmazione, la scelta è ricaduta su Python, dato che rappresenta lo standard per l'utilizzo di Flower e semplifica l'impiego di modelli e moduli adatti al machine learning.

Come rete neurale è stata scelta InceptionResnetV1 senza pesi pre-addestrati. Questa è adatta a contenere modelli in grado di trasformare immagini di volti in vettori numerici, chiamati embeddings, che catturano le caratteristiche distintive di questi. L'architettura di tale rete combina i principi delle reti Inception con quelli delle ResNet (Residual Networks). Le prime permettono di elaborare convoluzioni di diverse dimensioni dello stesso layer (ad esempio 1×1 , 3×3 , 5×5), in modo tale da catturare sia dettagli che pattern più globali senza aumentare eccessivamente il numero di parametri. Le ResNet, come visto in precedenza, consentono di evitare alcuni problemi nell'addestramento di reti profonde tramite l'introduzione del concetto di residual block.

Per l'addestramento e la valutazione del modello è stato utilizzato il dataset Labeled Faces in the Wild (LFW). LFW è costituito da oltre 13.000 immagini di circa 5.700 persone provenienti da internet che rappresentano volti in condizioni reali, con variazioni di illuminazione, espressione, posa e sfondo. È ampiamente riconosciuto nella comunità di ricerca ed è diventato uno standard di riferimento per il confronto delle prestazioni di modelli di face recognition.



Figura 3.2: Campioni di volti provenienti da LFW [39].

Il lato server viene gestito all'interno del file `server_app.py`, dove vi è la creazione di un'istanza di `ServerApp`. Il costruttore di questa prende in ingresso una funzione `server_fn` adatta a specificare e configurare il funzionamento del server centrale, dalla scelta del modello alla strategia di aggregazione. InceptionResnetV1 viene inizializzato tramite il costruttore dell'omonima classe, facente parte della libreria `facenet_pytorch`. Per il framework la scelta è dunque ricaduta su PyTorch, il quale rappresenta lo standard per lo sviluppo sperimentale in ambito accademico, e la documentazione di Flower lo preferisce rispetto ad altre librerie. PyTorch è stato scelto anche per la sua proprietà di calcolo dei grafi in maniera dinamica. Un grafo computazionale consiste

in una rappresentazione astratta di un calcolo come insieme di nodi e archi. Questa viene utilizzata nell'addestramento di una rete neurale per mantenere il flusso di dati e operazioni matematiche che vengono eseguite tra i vari layer. Dei grafi dinamici, a differenza di quelli statici, non vengono costruiti a priori, ma generati durante l'esecuzione. Questo porta a un'ottimizzazione minore in termini di performance dell'addestramento, infatti, framework come TensorFlow vengono spesso preferiti in scenari di produzione. Nella ricerca invece la maggiore flessibilità e facilità di debugging rende PyTorch la scelta naturale nella maggioranza dei casi. Una volta ottenuto il modello, da questo vengono estratti i pesi che, da una lista di array NumPy, vengono trasformati in un oggetto `Parameters` tramite una funzione di utilità di Flower. Questo è infatti il formato standard utilizzato dal framework per scambiare i pesi tra server e client durante l'addestramento.

Per quanto riguarda la strategia di aggregazione è stata realizzata una classe apposita `FedAvgSaving` che eredita da `FedAvg`, ovvero il tipo fornito da Flower per l'algoritmo Federated Averaging discusso in precedenza. L'estensione creata lascia invariata la logica di `FedAvg` aggiungendo una funzionalità di salvataggio del modello globale al termine dell'addestramento federato. In questo modo è possibile ottenere una copia locale dei pesi e valutarne le prestazioni tramite il modulo apposito.

```

1  class FedAvgSaving(FedAvg):
2      def __init__(self, num_rounds, *args, **kwargs):
3          super().__init__(*args, **kwargs)
4          self.num_rounds = num_rounds
5
6      def aggregate_fit(self, server_round, results, failures):
7          # aggregation
8          aggregated_parameters, aggregated_metrics = super().aggregate_fit(
9              server_round, results, failures
10         )
11
12         # last round -> save model
13         if aggregated_parameters is not None and server_round ==
14             ↪ self.num_rounds:
15             save_global_model(aggregated_parameters)
16
17         return aggregated_parameters, aggregated_metrics
18
19 def save_global_model(parameters):
20     ndarrays = parameters_to_ndarrays(parameters)
21     model = InceptionResnetV1()
22
23     state_dict = dict(zip(model.state_dict().keys(),
24                           [torch.tensor(nd) for nd in ndarrays]))
25     model.load_state_dict(state_dict, strict=True)
26
27     torch.save(model.state_dict(), "global_model.pth")
28     print("Modello globale salvato su 'global_model.pth'")

```

La parte relativa alla gestione dei client è stata inserita all'interno del file `client_app.py`, con alcune funzioni di servizio contenute all'interno di `task.py`. Qui, analogamente a

ciò che accade lato server, viene istanziato un oggetto di tipo `ClientApp` al quale si passa la funzione `client_fn` adatta all'estrazione dei parametri di training locale e alla creazione di un'istanza `FlowerClient`. È qui che viene caricato il dataset di allenamento, il quale viene suddiviso in partizioni IID tra i vari client. Ciascuna partizione viene poi divisa a sua volta in un insieme da utilizzare per il training e un altro per la valutazione locale, con rapporto 80/20. Essendo il dataset estratto da HuggingFace e avendo qui una struttura con gli attributi `image` e `filename`, tramite codice si creano degli indici adatti a rappresentare le label del dataset, ossia ciascuna persona unica presente. Successivamente ogni immagine viene ridimensionata a 160x160 pixel, convertita in un tensore PyTorch e i propri valori vengono normalizzati da $[0,255]$ a $[0,1]$, in maniera tale da poter essere compatibile con il modello InceptionResnetV1. Le strutture ottenute vengono inserite all'interno di due `DataLoader`, uno di training e l'altro di testing, così da poter iterare il dataset batch dopo batch. Questo è fondamentale per diversi aspetti:

1. L'intero dataset, essendo molto grande, potrebbe non entrare in memoria RAM tutto insieme.
2. La divisione consente di calcolare la loss per ogni singolo batch, invece di aggiornare i pesi sul dataset intero. In questo modo è possibile utilizzare l'ottimizzazione stocastica e trovare minimi migliori nella funzione di perdita.
3. Tramite l'opzione `shuffle = True` di `DataLoader` i dati vengono mescolati, aiutando il modello a non sovradattarsi a un ordine specifico di questi.

```

1 dataset = None
2
3 def load_data(partition_id: int, num_partitions: int):
4     """
5     Load dataset and create trainloader and testloader.
6     """
7     # only initialize FederatedDataset once
8     global dataset
9     if dataset is None:
10         partitioner = IidPartitioner(num_partitions=num_partitions)
11         dataset = FederatedDataset(
12             dataset="bitmind/lfw",
13             partitioners={"train": partitioner},
14         )
15
16     partition = dataset.load_partition(partition_id)
17
18     # divide data on each node: 80% train, 20% test
19     partition_train_test = partition.train_test_split(test_size=0.2, seed=42)
20
21     # mapping person name -> idx (label)
22     all_persons = [extract_person(fname) for fname in
23                     ↪ partition_train_test["train"]["filename"] +
24                     ↪ partition_train_test["test"]["filename"]]
25
26     unique_persons = sorted(set(all_persons))
27     person_to_idx = {name: idx for idx, name in enumerate(unique_persons)}

```

```

25
26     # transform data for compatibility with the model
27     pytorch_transforms = Compose(
28         [Resize((160, 160)), ToTensor(), Normalize((0.5, 0.5, 0.5), (0.5,
29             ↪ 0.5, 0.5))]
30     )
31     def apply_transforms(batch):
32         batch["image"] = torch.stack([pytorch_transforms(img) for img in
33             ↪ batch["image"]])
34         batch["label"] = torch.tensor([person_to_idx[extract_person(fname)]
35             ↪ for fname in batch["filename"]], dtype=torch.long)
36         return batch
37     partition_train_test =
38         ↪ partition_train_test.with_transform(apply_transforms)
39
40     trainloader = DataLoader(partition_train_test["train"], batch_size=32,
41         ↪ shuffle=True)
42     testloader = DataLoader(partition_train_test["test"], batch_size=32)
43
44     return trainloader, testloader

```

Una volta preparate le partizioni per ciascun client queste vengono passate al costruttore di `FlowerClient`, classe che gestisce il comportamento dei dispositivi terminali durante il loro ciclo di vita, in particolare durante le fasi di allenamento e valutazione locale. All'interno della funzione `fit` viene recuperato il modello globale caricando i pesi comunicati dal server. Una volta fatto ciò, avvengono le epoche di training locale vero e proprio, che hanno l'obiettivo di ottimizzare i pesi del modello affinché le predizioni siano il più possibile coerenti con le etichette reali. Come funzione di perdita la scelta è ricaduta sulla Cross-Entropy Loss [40], ampiamente utilizzata per problemi di classificazione multi-classe. Date C classi, una predizione del modello sotto forma di distribuzione di probabilità $\hat{y}$ e un'etichetta y che stabilisce l'output corretto, la cross-entropy loss è definita nel seguente modo:

$$\mathcal{L}_{CE} = - \sum_{c=1}^C y_c \log(\hat{y}_c) \quad (3.3)$$

dove y_c vale 1 se c è la classe corretta e 0 altrimenti, e $\hat{y}_c$ è la probabilità secondo cui il modello prevede che il volto appartenga alla classe c . La funzione misura quanto la previsione del modello si discosta dalla distribuzione reale delle etichette, penalizzando maggiormente predizioni molto lontane dalla classe corretta e spingendo quindi il modello a generare probabilità alte solo per la classe giusta. L'utilizzo della classe `CrossEntropyLoss` di PyTorch consente di combinare il calcolo di $\hat{y}_c$ e $\log$ in un'unica operazione numericamente stabile, evitando possibili underflow e overflow.

Come ottimizzatore è stato scelto Adam [41], il quale utilizza la discesa del gradiente stocastico (SGD) e aggiunge altre funzionalità. La SGD svolge la minimizzazione della funzione di perdita aggiornando i pesi nella direzione opposta al gradiente della loss. Questo viene fatto utilizzando un singolo campione o un piccolo batch di campioni alla volta, invece dell'intero dataset, in quanto altrimenti risulterebbe troppo costoso e

lento. A ciò Adam aggiunge i concetti del momentum adattivo e adattamento del learning rate per parametro. Il primo consiste nel mantenere una media esponenziale dei gradienti passati, in maniera tale da non aggiornare i pesi solo sulla base del gradiente corrente e smorzando così eventuali oscillazioni durante l'ottimizzazione. L'adattamento del learning rate funziona invece tramite il mantenimento di una stima della varianza dei gradienti, la quale permette di aggiornare ciascun peso in maniera proporzionale alla sua incertezza. In questo modo gradienti molto grandi o molto piccoli non dominano l'ottimizzazione e la convergenza viene resa più stabile, riducendo la necessità di scegliere con grande precisione il learning rate.

Nel codice viene utilizzato un ciclo che scorre le epoche, e all'interno di queste il dataset viene passato batch per batch. `Model(images)` genera le predizioni del modello, `criterion(outputs, labels)` calcola la loss nella maniera vista in precedenza, `loss.backward()` calcola i gradienti rispetto ai pesi del modello e `optimizer.step()` utilizza Adam per aggiornarli.

```

1  def train(model, trainloader, epochs, device, lr=1e-4):
2      """
3      Train model on a dataset.
4      """
5      model.train()
6
7      criterion = nn.CrossEntropyLoss()
8      optimizer = optim.Adam(model.parameters(), lr=lr)
9
10     epoch_losses = []
11
12     for epoch in range(epochs):
13         print(f"Training epoch {epoch+1}/{epochs} started")
14
15         running_loss = 0.0 # loss sum
16         total = 0 # num. total images
17
18         for batch in trainloader:
19             images = batch["image"].to(device)
20             labels = batch["label"].to(device)
21
22             optimizer.zero_grad() # reset gradients from previous batch
23             outputs = model(images) # generate logits
24             loss = criterion(outputs, labels) # calculate loss
25             loss.backward() # calculate gradients
26             optimizer.step() # update model weights based on the loss
27
28             running_loss += loss.item() * images.size(0)
29             total += images.size(0)
30
31             # clean memory on MPS
32             del images, labels, outputs, loss
33             if device.type == "mps":
34                 torch.mps.empty_cache()

```



```

35
36     epoch_loss = running_loss / total
37     epoch_losses.append(epoch_loss)
38
39     print(f"Training epoch {epoch+1}/{epochs} finished - Loss:
      ↪ {epoch_loss:.4f}")
40
41     return sum(epoch_losses) / len(epoch_losses) # avg loss on all epochs

```

La funzione `evaluate` di `FlowerClient` svolge invece la valutazione del modello aggiornato localmente. Per generare le metriche Top-1 e Top-k Accuracy viene eseguito il seguente processo:

1. Estrazione degli embeddings. Ogni immagine del dataset viene fornita in input al modello aggiornato ottenendo l'embedding corrispondente.
2. Calcolo dell'embedding media per persona. Siccome ogni persona viene solitamente rappresentata da più immagini nel dataset, è possibile calcolare un embedding medio che funge da firma di tale persona.
3. Calcolo delle metriche. Viene valutata l'accuratezza del modello confrontando, per ciascuna immagine del dataset, l'embedding corrispondente con gli embeddings medi di ciascuna persona, ottenendo una classifica delle identità più simili.

L'estrazione degli embeddings viene svolta con l'opzione `torch.no_grad()`, in maniera tale da non utilizzare il tracciamento dei gradienti, non necessario in questa fase, e accelerare l'esecuzione. Inoltre, le immagini vengono elaborate in batch, così da utilizzare la memoria in maniera efficiente. Il calcolo dell'embedding medio avviene invece identificando prima tutte le persone presenti nel dataset tramite la funzione `np.unique()` richiamata sulle labels. Successivamente, viene ciclata la lista di persone ottenuta e, per ciascuna di queste, viene richiamata la funzione `np.mean()` per svolgere la media tra i valori presenti nell'array di embeddings relativo. Il risultato ottenuto sarà utile nell'ultima fase del processo di valutazione, quella che comprende il calcolo delle metriche. Qui, siccome gli embedding sono dei vettori numerici, viene utilizzata la similarità coseno per valutare la vicinanza in termini di orientamento nello spazio con tutti gli embedding medi. La similarità coseno tra due vettori u e v si calcola nella seguente maniera:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \quad (3.4)$$

Il risultato di questa espressione è un numero tra -1 e 1, con valori più vicini a 1 che indicano una maggiore somiglianza angolare tra i vettori. La principale motivazione nella scelta della similarità coseno è data dal fatto che gli embeddings prodotti dai modelli di riconoscimento facciale possono avere moduli diversi a causa di variazioni di luminosità, contrasto o normalizzazione. Altre metriche come la distanza Euclidea, la quale misura la distanza lineare nello spazio vettoriale, verrebbero influenzate dalle differenze di scala tra gli embeddings, rendendo la misura meno robusta a variazioni non informative. Una volta calcolata la similarità tra i vettori e ottenuta una classifica delle persone più simili, è possibile stabilire la correttezza Top-1 e Top-k controllando rispettivamente la prima e le prime k posizioni. L'accuratezza viene poi calcolata come il rapporto tra il numero di predizioni corrette e il numero totale di immagini di test.

```

1  def extract_embeddings(model, dataloader, device):
2      """
3      Extract embeddings for each image in a dataset.
4      """
5      all_embeddings = []
6      all_labels = []
7
8      with torch.no_grad():
9          for batch in dataloader:
10             images = batch["image"].to(device)
11             labels = batch["label"]
12
13             embeddings = model(images)
14
15             all_embeddings.append(embeddings.cpu().numpy())
16             all_labels.extend(labels.cpu().numpy())
17
18             # clean memory on MPS
19             del images, embeddings
20             if device.type == "mps":
21                 torch.mps.empty_cache()
22
23     return np.vstack(all_embeddings), np.array(all_labels)
24
25 def calculate_person_embeddings(embeddings, labels):
26     """
27     Calculate average embedding for each person.
28     """
29     unique_labels = np.unique(labels)
30     person_embeddings = {}
31
32     for person_id in unique_labels:
33         person_mask = labels == person_id
34         person_imgs = embeddings[person_mask]
35         person_embeddings[person_id] = np.mean(person_imgs, axis=0)
36
37     return person_embeddings
38
39 def calculate_recognition_accuracy_topk(embeddings, labels,
40     ↪ person_embeddings, k=5):
41     """
42     Calculate recognition accuracy of the model for the single person and top
43     ↪ k
44     """
45     correct_top1 = 0
46     correct_topk = 0
47     total = len(embeddings)
48
49     person_ids = list(person_embeddings.keys())
50     person_emb_matrix = np.array([person_embeddings[p] for p in person_ids])

```



```

49
50     for test_embedding, true_label in zip(embeddings, labels):
51         sims = cosine_similarity([test_embedding], person_emb_matrix)[0]
52
53         # top 1
54         predicted_person_top1 = person_ids[np.argmax(sims)]
55         if predicted_person_top1 == true_label:
56             correct_top1 += 1
57
58         # top k
59         topk_indices = np.argsort(sims)[::-1][:k]
60         if true_label in [person_ids[idx] for idx in topk_indices]:
61             correct_topk += 1
62
63     accuracy_top1 = correct_top1 / total
64     accuracy_topk = correct_topk / total
65     return accuracy_top1, accuracy_topk
66
67 def test(model, testloader, device, k):
68     """
69     Test model on a dataset and calculate accuracy.
70     """
71     model.eval()
72
73     embeddings, labels = extract_embeddings(model, testloader, device)
74     person_embeddings = calculate_person_embeddings(embeddings, labels)
75
76     accuracy_top1, accuracy_topk =
77     ↪ calculate_recognition_accuracy_topk(embeddings, labels,
78     ↪ person_embeddings, k=k)
79
80     return {
81         "accuracy_top1": accuracy_top1,
82         "accuracy_topk": accuracy_topk
83     }

```

Le variabili per l'addestramento federato sono contenute all'interno del file di configurazione `pyproject.toml`, in cui troviamo:

- **num-server-rounds**: numero di round di addestramento coordinati dal server.
- **options.num-supernodes**: numero di nodi complessivi dai quali poter scegliere il sottoinsieme di client utili al singolo round di addestramento.
- **fraction-fit**: frazione di client selezionati tra i supernodes per partecipare all'addestramento.
- **local-epochs**: numero di epoche di addestramento locale che ciascun client esegue prima di inviare gli aggiornamenti al server.

Una volta configurati questi parametri è possibile avviare il sistema di federated learning tramite il comando da terminale `flwr run [nome_app]`.

Infine, una volta terminato l'addestramento, si ottiene il file `global_model.pth` contenente il valore dei pesi del modello globale. Questo viene utilizzato dal modulo di testing per calcolare la Top-1 e Top-k Accuracy tramite una chiamata alle funzioni analizzate in precedenza.

3.4 Risultati sperimentali e sviluppi futuri

La simulazione realizzata è stata eseguita in maniera sperimentale su un MacBook Air M2, utilizzando la libreria Metal Performance Shaders (MPS) per sfruttare la GPU integrata nel chip M2 e velocizzare l'addestramento. Il modello iniziale, non pre-addestrato, è stato messo alla prova fornendo i seguenti risultati:

- Top-1 Accuracy: 0.3757 (37.57%)
- Top-5 Accuracy: 0.4133 (41.33%)

Utilizzando questa base di partenza è stato svolto l'addestramento federato con i seguenti parametri:

- `num-server-rounds`: 50.
- `options.num-supernodes`: 5.
- `fraction-fit`: 0.7.
- `local-epochs`: 1.

I 50 round di addestramento sono terminati in 1 ora, 53 minuti e 17 secondi e hanno generato un modello globale con le seguenti metriche:

- Top-1 Accuracy: 0.7035 (70.35%)
- Top-5 Accuracy: 0.7796 (77.96%)

Questi risultati mostrano un miglioramento significativo rispetto al modello iniziale non addestrato, dimostrando l'efficacia del federated learning nell'ambito del riconoscimento facciale. Va sottolineato che queste prestazioni sono state ottenute all'interno di un contesto sperimentale, caratterizzato da risorse limitate che hanno obbligato alla scelta di un numero basso di supernodi e di un dataset limitato come quello di LFW. Nonostante ciò, le metriche ottenute sono paragonabili a quelle dello stesso modello pre-addestrato su un dataset molto più ampio come Casia WebFace, che contiene 494.414 immagini. Questo infatti, messo alla prova sulle stesse funzioni di test ha generato questi risultati: Top-1 Accuracy 0.7510 (75.10%), Top-5 Accuracy 0.8481 (84.81%).

Durante l'esecuzione dell'allenamento federato è stata svolta anche un'attività di monitoraggio delle risorse utilizzate dai diversi client. Per ciascuno di questi, la CPU occupata variava tipicamente tra il 25% e il 47%, con picchi occasionali che non superavano mai il 50%. Questo è dato principalmente dall'utilizzo della libreria MPS, che permette un uso combinato di CPU e GPU, eseguendo su quest'ultima la maggior

parte dei calcoli più intensivi, come convoluzioni e operazioni di backpropagation. La memoria RAM utilizzata dai diversi processi Python relativi ai dispositivi edge oscillava tra 2.5 GB e 3.5 GB, variando significativamente tra i vari client. Questo mostra come Flower, pur coordinando il training federato, consenta a ciascun dispositivo simulato di utilizzare le risorse in modo indipendente, evitando saturazioni e colli di bottiglia locali. È stato inoltre notato come alcuni processi abbiano ridotto drasticamente il loro utilizzo di CPU e memoria in alcuni momenti durante l'addestramento, mostrando valori prossimi allo zero. Ciò è dato dal Virtual Client Engine di Flower, il quale gestisce l'attività di ciascun client in maniera dinamica avviando l'addestramento locale solo quando le risorse di sistema sono disponibili.

Aver dimostrato il funzionamento del federated learning su un tale caso d'uso lascia aperte molte strade per sviluppi futuri. Una prima opzione consiste nell'utilizzo di architetture più complesse e GPU dedicate per consentire un aumento dei client e dei round di addestramento, al fine di valutare se il modello riesca a convergere ulteriormente. Un'altra possibilità è quella di lavorare sul dataset, utilizzandone di più complessi e realistici come VGGFace2, e introdurre tecniche di data augmentation per incrementare la variabilità dei dati locali e migliorare la capacità di generalizzazione del modello. Così facendo sarà possibile sperimentare anche algoritmi di aggregazione alternativi al FedAvg, capaci di ridurre la sensibilità alla distribuzione non-IID dei dati forniti. Infine sarebbe interessante implementare algoritmi adatti alla tutela della privacy per ridurre gli attacchi e impedire la ricostruzione dei dati locali, svolgendo anche uno studio sul trade-off tra sicurezza e performance.

Conclusione

Il presente lavoro di tesi ha esplorato l'applicazione del machine learning su dispositivi edge prima tramite una trattazione teorica e poi con un progetto pratico. Questo ha riguardato la realizzazione di un sistema di apprendimento automatico distribuito per la calibrazione dei pesi di un modello di riconoscimento facciale utilizzando il framework Flower. L'obiettivo principale di un tale progetto era quello di dimostrare la fattibilità tecnica di questo approccio, valutarne le prestazioni e fornire una base di partenza modulare per ampliamenti e sviluppi futuri.

La validità del paradigma di federated learning è stata dimostrata attraverso i risultati ottenuti, dato che il modello InceptionResnetV1 inizialmente non addestrato (Top-1 Accuracy: 37.57%, Top-5 Accuracy: 41.33%), ha mostrato un ampio miglioramento dopo 50 round di addestramento, raggiungendo una Top-1 Accuracy del 70.35% e una Top-5 Accuracy del 77.96%. I vantaggi del federated learning sono stati resi chiari dall'implementazione adottata. La struttura modulare ha permesso infatti di evidenziare la separazione tra client e server e il conseguente mantenimento di un alto livello di privacy locale. Il monitoraggio delle attività e delle risorse di sistema ha permesso anche di comprendere in maniera pratica il funzionamento del framework Flower e come questo sfrutti in maniera intelligente l'allocazione dinamica della memoria.

Il sistema è stato inoltre eseguito in un ambiente sperimentale caratterizzato da limitazioni in termini di risorse computazionali, costringendo l'utilizzo di 5 soli supernodi e un dataset di dimensioni limitate. Ciò ha messo in luce molteplici prospettive di sviluppo futuro, tra cui la sperimentazione in un contesto con hardware più potente, in maniera tale da spingere al limite tale approccio e raggiungere la convergenza del modello. Altre numerose sfide troverebbero approfondimento in future analisi, spaziando dagli aspetti legati alla sicurezza alla natura eterogenea dei dati utilizzati. L'implementazione di tecniche di aggregazione alternative e algoritmi di cybersecurity applicati al trasferimento dei parametri tra client e server offre un ampio spettro di possibilità di ricerca.

In conclusione, questo progetto ha voluto sperimentare con un recente cambio di paradigma nell'ambito dell'apprendimento automatico, in maniera tale da comprenderne a fondo il funzionamento e le potenzialità. Il lavoro svolto costituisce una base solida per implementazioni e ricerche future, in un settore tecnologico che ha il potenziale di assumere un ruolo chiave nel prossimo futuro dell'intelligenza artificiale.

Bibliografia

- [1] Iqbal H Sarker. Machine learning: Algorithms, real-world applications and research directions. *SN computer science*, 2(3):160, 2021.
- [2] Santiago Badillo, Bálint Banfai, Fabian Birzele, and et al. An introduction to machine learning. *Clinical Pharmacology & Therapeutics*, 107(4):871–885, 2020.
- [3] Daniel Sáez Trigueros, Li Meng, and Margaret Hartnett. Face recognition: From traditional to deep learning methods. *arXiv preprint arXiv:1811.00116*, 2018.
- [4] Matthew Turk and Alex Pentland. Eigenfaces for recognition. *Journal of cognitive neuroscience*, 3(1):71–86, 1991.
- [5] Peter N. Belhumeur, Joao P Hespanha, and David J. Kriegman. Eigenfaces vs. fisherfaces: Recognition using class specific linear projection. *IEEE Transactions on pattern analysis and machine intelligence*, 19(7):711–720, 1997.
- [6] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 2002.
- [7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [8] Haochen Hua, Yutong Li, Tonghe Wang, Nanqing Dong, Wei Li, and Junwei Cao. Edge computing with artificial intelligence: A machine learning perspective. *ACM Comput. Surv.*, 55(9), January 2023.
- [9] Xukan Ran, Haolanz Chen, Xiaodan Zhu, Zhenming Liu, and Jiasi Chen. Deep-decision: A mobile deep learning framework for edge video analytics. In *IEEE INFOCOM 2018-IEEE conference on computer communications*, pages 1421–1429. IEEE, 2018.
- [10] Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor Mudge, Jason Mars, and Lingjia Tang. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. *ACM SIGARCH Computer Architecture News*, 45(1):615–629, 2017.
- [11] Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. Branchynet: Fast inference via early exiting from deep neural networks. *arXiv preprint arXiv:1709.01686*, 2017.

- [12] Utsav Drolia, Katherine Guo, and Priya Narasimhan. Precog: Prefetching for image recognition applications at the edge. In *Proceedings of the Second ACM/IEEE Symposium on Edge Computing*, pages 1–13, 2017.
- [13] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*, pages 1–12, 2017.
- [14] Zidong Du, Robert Fasthuber, Tianshi Chen, Paolo Ienne, Ling Li, Tao Luo, Xiaobing Feng, Yunji Chen, and Olivier Temam. Shidiannao: Shifting vision processing closer to the sensor. In *Proceedings of the 42nd annual international symposium on computer architecture*, pages 92–104, 2015.
- [15] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.
- [16] Praneeth Vepakomma, Otkrist Gupta, Tristan Swedish, and Ramesh Raskar. Split learning for health: Distributed deep learning without sharing raw patient data. *arXiv preprint arXiv:1812.00564*, 2018.
- [17] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on knowledge and data engineering*, 22(10):1345–1359, 2009.
- [18] Gido M Van de Ven, Tinne Tuytelaars, and Andreas S Tolias. Three types of incremental learning. *Nature Machine Intelligence*, 4(12):1185–1197, 2022.
- [19] Timothy Hospedales, Antreas Antoniou, Paul Micaelli, and Amos Storkey. Meta-learning in neural networks: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 44(9):5149–5169, 2021.
- [20] Léon Bottou. Stochastic gradient descent tricks. In *Neural networks: tricks of the trade: second edition*, pages 421–436. Springer, 2012.
- [21] Enmao Diao, Jie Ding, and Vahid Tarokh. Heteroff: Computation and communication efficient federated learning for heterogeneous clients. *arXiv preprint arXiv:2010.01264*, 2020.
- [22] Bingyan Liu, Nuoyan Lv, Yuanchun Guo, and Yawen Li. Recent advances on federated learning: A systematic survey. *Neurocomputing*, 597:128019, 2024.
- [23] Fan Lai, Xiangfeng Zhu, Harsha V Madhyastha, and Mosharaf Chowdhury. Oort: Efficient federated learning via guided participant selection. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, pages 19–35, 2021.
- [24] Chenning Li, Xiao Zeng, Mi Zhang, and Zhichao Cao. Pyramidfl: A fine-grained client selection framework for efficient federated learning. In *Proceedings of the 28th annual international conference on mobile computing and networking*, pages 158–171, 2022.

- [25] Cynthia Dwork. Differential privacy. In *International colloquium on automata, languages, and programming*, pages 1–12. Springer, 2006.
- [26] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pages 169–178, 2009.
- [27] Chengliang Zhang, Suyi Li, Junzhe Xia, Wei Wang, Feng Yan, and Yang Liu. {BatchCrypt}: Efficient homomorphic encryption for {Cross-Silo} federated learning. In *2020 USENIX annual technical conference (USENIX ATC 20)*, pages 493–506, 2020.
- [28] Victor Costan and Srinivas Devadas. Intel sgx explained. *Cryptology ePrint Archive*, 2016.
- [29] Fan Mo, Hamed Haddadi, Kleomenis Katevas, Eduard Marin, Diego Perino, and Nicolas Kourtellis. Ppfl: Privacy-preserving federated learning with trusted execution environments. In *Proceedings of the 19th annual international conference on mobile systems, applications, and services*, pages 94–108, 2021.
- [30] Daniel J Beutel, Taner Topal, Akhil Mathur, Xinchu Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque De Gusmão, et al. Flower: A friendly federated learning research framework. *arXiv preprint arXiv:2007.14390*, 2020.
- [31] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017.
- [32] Kwing Hei Li, Pedro Porto Buarque de Gusmão, Daniel J Beutel, and Nicholas D Lane. Secure aggregation for federated learning in flower. In *Proceedings of the 2nd ACM International Workshop on Distributed Machine Learning*, pages 8–14, 2021.
- [33] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. Ray: A distributed framework for emerging {AI} applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*, pages 561–577, 2018.
- [34] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- [35] Thien Duc Nguyen, Samuel Marchal, Markus Miettinen, Hossein Fereidooni, Natarajah Asokan, and Ahmad-Reza Sadeghi. Diot: A federated self-learning anomaly detection system for iot. In *2019 IEEE 39th International conference on distributed computing systems (ICDCS)*, pages 756–767. IEEE, 2019.

- [36] Thien Duc Nguyen, Samuel Marchal, Markus Miettinen, Hossein Fereidooni, Nadarajah Asokan, and Ahmad-Reza Sadeghi. Diot: A federated self-learning anomaly detection system for iot. In *2019 IEEE 39th International conference on distributed computing systems (ICDCS)*, pages 756–767. IEEE, 2019.
- [37] Li Huang, Andrew L Shea, Huining Qian, Aditya Masurkar, Hao Deng, and Dianbo Liu. Patient clustering improves efficiency of federated machine learning to predict mortality and hospital stay time using distributed electronic medical records. *Journal of biomedical informatics*, 99:103291, 2019.
- [38] Yang Liu, Anbu Huang, Yun Luo, He Huang, Youzhi Liu, Yuanyuan Chen, Lican Feng, Tianjian Chen, Han Yu, and Qiang Yang. Fedvision: An online visual object detection platform powered by federated learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 13172–13179, 2020.
- [39] Tianyue Zheng, Weihong Deng, and Jiani Hu. Cross-age lfw: A database for studying cross-age face recognition in unconstrained environments. *arXiv preprint arXiv:1708.08197*, 2017.
- [40] Anqi Mao, Mehryar Mohri, and Yutao Zhong. Cross-entropy loss functions: Theoretical analysis and applications. In *International conference on Machine learning*, pages 23803–23828. pmlr, 2023.
- [41] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [42] Michael I Jordan and Tom M Mitchell. Machine learning: Trends, perspectives, and prospects. *Science*, 349(6245):255–260, 2015.
- [43] Kajaree Das and Rabi Narayan Behera. A survey on machine learning: concept, algorithms and applications. *International Journal of Innovative Research in Computer and Communication Engineering*, 5(2):1301–1309, 2017.
- [44] Yassine Kortli, Mounir Jridi, Ali Al Falou, and Mohamed Atri. Face recognition systems: A survey. *Sensors*, 2020.
- [45] Mei Wang and Weihong Deng. Deep face recognition: A survey. *Neurocomputing*, 429:215–244, 2021.
- [46] Aymen Rayane Khouas, Mohamed Reda Bouadjenek, Hakim Hacid, and Sunil Aryal. Training machine learning models at the edge: A survey. *arXiv preprint arXiv:2403.02619*, 2024.
- [47] En Li, Liekang Zeng, Zhi Zhou, and Xu Chen. Edge ai: On-demand accelerating deep neural network inference via edge computing. *IEEE Transactions on Wireless Communications*, 19(1):447–457, 2020.
- [48] Jiasi Chen and Xukan Ran. Deep learning with edge computing: A review. *Proceedings of the IEEE*, 107(8):1655–1674, 2019.

- [49] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. Federated learning: Challenges, methods, and future directions. *IEEE signal processing magazine*, 37(3):50–60, 2020.
- [50] Jurgen Schmidhuber. Multi-column deep neural networks for image classification. In *Proceedings of the 2012 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3642–3649, 2012.